NEUROPLAST - A SELF-CONSOLIDATING RL AGENT

DSN4091-CAPSTONE PROJECT PHASE-I

Phase – I Report

Submitted by

Vaibhav Tiwari (23BAI10108)

Kavish Bishnoi (23BAI10789)

Kushal Upadhyay (23BAI10259)

Yash Yadav (23BAI10484)

Adamya Upadhyay (23BAI11221)

Siddharth Ubbott (23BAI10997)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY SEHORE,

MADHYA PRADESH - 466114

October 2026

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled **"NEUROPLAST - A SELF-CONSOLIDATING RL AGENT"** is the bonafide work of **"Vaibhav Tiwari (23BAI10108), Kavish Bishnoi (23BAI10789), Kushal Upadhyay (23BAI10259), Yash Yadav (23BAI10484), Adamya Upadhyay (23BAI11221), Siddharth Ubbott (23BAI10997)"** who carried out the project work (DSN4091- Capstone Project Phase-I) under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR
Dr. Pradeep Kumar Mishra
School of Computing Science Engineering and Artificial Intelligence
VIT BHOPAL UNIVERSITY

PROJECT GUIDE
Dr. Trapti Sharma
[TBD: guide designation]
School of Computing Science Engineering and Artificial Intelligence
VIT BHOPAL UNIVERSITY

The DSN4091-Capstone Project Phase-I Viva Voce Examination is held on October 12, 2026

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Lord Almighty for his presence and immense blessings throughout the project work.

We would like to thank our internal guide Dr. Trapti Sharma, for continually guiding and actively participating in our project, and giving valuable suggestions to complete the project works.

We wish to express our heartfelt gratitude to Dr. Pradeep Kumar Mishra, PC-Lead, School of Computing Science Engineering and Artificial Intelligence for much of his valuable support and encouragement in carrying out this work.

We wish to express our heartfelt gratitude to Dr Pon Harshavardhanan, Dean, School of Computing Science Engineering and Artificial Intelligence for much of his valuable support and encouragement in carrying out this work.

We would like to thank all the technical and teaching staff of the School of Computing Science Engineering and Artificial Intelligence, who extended directly or indirectly all support.

Last, but not least, we are deeply indebted to our parents who have been the greatest support while we worked day and night for the project to make it a success.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Architecture of the agent: environment, spiking encoder, Transformer working memory and actor–critic heads	15
4.2	Continual training of the self-consolidating variant: wake and sleep phases alternate on the same shared trunk, linked by the replay buffer	15
4.3	Dashboard: headline, plain-language summary and the tally of result groups (light theme)	16
4.4	Dashboard: forgetting curves with a sequence selector; shaded bands are 95% confidence intervals	17
6.1	Local-plasticity arms on DoorKey-6x6 (10 seeds per arm). Dots are seeds' AUC (filled: solved); bars are means with 95% confidence intervals; numbers above the dots are seeds solved. Left: backprop as the learning signal; right: DFA with and without homeostasis	24
6.2	Final accuracy against total memory (fp32 parameters plus replay buffer, log scale). Numbers above the axis are replay states stored per task; the 5-task SNN runs use 450k frames per task	25
6.3	Post-freeze follow-up (CNN, 5 tasks, 10 seeds per arm): mean final accuracy on a 0.85–1.00 scale, with memory	25
6.4	Accuracy against operation count for the spiking and the conventional encoder (behaviour cloning of a DoorKey-6x6 teacher, 2 seeds per point). Left: held-out action accuracy; right: closed-loop return of the cloned policy	26
6.5	Forgetting on the continual task sequences. Top: return on the first task after each task is trained; bottom: mean return over the tasks seen so far. Means over seeds with 95% confidence intervals (clipped to [0, 1])	28
6.6	Sleep versus replay at an exactly matched replay budget (SNN+Transformer, fetch3; 8 seeds per arm). Grey lines connect runs with the same seed. Left: final accuracy; right: forgetting	29
6.7	Memory-benchmark probes on MiniGrid-Memory S11 and S13 (CNN, one seed each): single frame versus a stack of the last 12 frames. A return of 0.5 is chance	30
A.1	Dashboard: mechanism scorecard with result groups, key numbers and evidence labels	37
A.2	Dashboard: the naive and sleep agents after the full fetch3 sequence, with recorded episodes and accuracy matrices	38
A.3	Dashboard: memory trade-off, with the key result and the post-freeze follow-up	39
A.4	Dashboard: the “What's next” section (plans, not results)	40
A.5	Final frames of the three recorded episodes per task, naive fine-tuning agent after the full sequence (the episode outcome is printed above each frame)	41
A.6	Final frames of the three recorded episodes per task, sleep agent after the full sequence	42

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Pros and cons of existing approaches to forgetting	6
3.1	Hardware used	8
3.2	Software stack: the requirement in requirements.txt and the installed version	9
3.3	Environments and task sequences	10
4.1	Mechanisms, implementations and controls (each row is switched on and off independently)	12
4.2	Repository layout	16
5.1	Training protocol (identical in every arm of a comparison)	18
6.1	Outcome by mechanism	23
6.2	Post-freeze follow-up: isolation at the shared trunk's memory (CNN, 5 tasks, seeds 201–210)	26
6.3	Specified local-plasticity contrasts on DoorKey-6x6 (10 seeds per arm; first arm minus second; Holm-corrected p in brackets)	27
6.4	Final ACC (mean ± SD over seeds) and FORGET on the continual sequences [exploratory; the matched row belongs to the confirmatory test]	28
6.5	Earlier positive findings that shrank or disappeared once a control was added	30
A.1	Accuracy matrix of the naive fine-tuning demo agent (seed 1): mean return on each task (columns) after training each task (rows), 100 evaluation episodes per cell	42
A.2	Accuracy matrix of the sleep demo agent (seed 1): mean return on each task (columns) after training each task (rows), 100 evaluation episodes per cell	43

ABSTRACT

A learning system trained on one task after another tends to lose what it learned earlier, a failure called catastrophic forgetting. Many proposed remedies borrow ideas from the brain, such as spiking neurons, local learning rules, homeostasis and a sleep-like offline phase in which the system consolidates what it has learned. They are usually added several at a time, so it is hard to tell which idea does the work. NeuroPlast is a small reinforcement-learning agent for grid-world tasks in which each of these mechanisms, including a sleep-like self-consolidation phase, can be switched on or off. We compared each mechanism with a matched control that is not brain-inspired but uses the same resources. The study ran on a 4-core CPU without a GPU (289 training runs, about 110 CPU hours). Each final comparison was specified in a version-controlled experiment log before its runs, with 3–10 random seeds per arm and 95% confidence intervals. No mechanism produced a statistically reliable improvement in final accuracy or forgetting beyond its matched control. With these seed counts, this rules out large effects only where the intervals are narrow, and it never shows that two methods are equivalent. In particular, self-consolidation during sleep did no better than ordinary replay at an equal replay budget. One narrow effect survived: homeostasis made a spiking network learn faster when its encoder learned without backpropagation. Two results are conditional: a shared network with a small memory buffer needed less memory than one network per task, but only against full-width per-task networks, and the spiking encoder needed fewer operations in an energy estimate while costing 5.2 times more CPU time to train. Whether Transformer working memory helps remains inconclusive. We also report seven earlier positive findings that shrank or disappeared once the controls were fair.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Figures	iv
	List of Tables	v
	Abstract	vi
1	**CHAPTER 1: PROJECT DESCRIPTION AND OUTLINE**	**1**
	1.1 INTRODUCTION	1
	1.2 MOTIVATION FOR THE WORK	2
	1.3 PROBLEM STATEMENT	2
	1.4 OBJECTIVE OF THE WORK	3
	1.5 SUMMARY	3
2	**CHAPTER 2: RELATED WORK INVESTIGATION**	**5**
	2.1 EXISTING APPROACHES/METHODS	5
	2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS	6
3	**CHAPTER 3: REQUIREMENT ARTIFACTS**	**8**
	3.1 INTRODUCTION	8
	3.2 HARDWARE AND SOFTWARE REQUIREMENTS	8
	3.3 SPECIFIC PROJECT REQUIREMENTS	9
	3.3.1 DATA REQUIREMENTS	9
	3.3.2 FUNCTIONAL REQUIREMENTS	10
	3.3.3 PERFORMANCE AND SECURITY REQUIREMENTS	11
	3.3.4 LOOK AND FEEL REQUIREMENTS	11
	3.4 SUMMARY	11
4	**CHAPTER 4: DESIGN METHODOLOGY AND ITS NOVELTY**	**12**
	4.1 METHODOLOGY AND GOAL	12
	4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS	13
	4.3 SOFTWARE ARCHITECTURAL DESIGNS	15
	4.4 USER INTERFACE DESIGNS	16
	4.5 SUMMARY	17
5	**CHAPTER 5: TECHNICAL IMPLEMENTATION AND ANALYSIS**	**18**
	5.1 OUTLINE	18
	5.2 TECHNICAL CODING AND CODE SOLUTIONS	18
	5.2.1 LIF NEURON AND SURROGATE GRADIENT	18
	5.2.2 STABILISED STDP AND HOMEOSTASIS	19
	5.2.3 THE MATCHED RANDOM-UPDATE CONTROL	19
	5.2.4 SLEEP PHASE WITH A MATCHED REPLAY BUDGET	20
	5.2.5 MEMORY ACCOUNTING	20
	5.2.6 STATISTICAL TESTS	20

CHAPTER NO.	TITLE	PAGE NO.
	5.2.7 EXPERIMENT QUEUE AND REPRODUCIBILITY PIPELINE	21
	5.3 PROTOTYPE SUBMISSION	22
	5.4 SUMMARY	22
6	**CHAPTER 6: PROJECT OUTCOME AND APPLICABILITY**	**23**
	6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM	23
	6.2 SIGNIFICANT PROJECT OUTCOMES	23
	6.2.1 SURVIVED: HOMEOSTASIS SPEEDS UP BACKPROP-FREE LEARNING	23
	6.2.2 CONDITIONAL: SHARED TRUNK AND SPIKING ENCODER	24
	6.2.3 DID NOT SURVIVE: STDP, HOMEOSTASIS WITH BACKPROP, SLEEP	26
	6.2.4 UNRESOLVED: TRANSFORMER WORKING MEMORY	29
	6.2.5 WHAT DID NOT SURVIVE CONTROLS	30
	6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS	31
	6.4 INFERENCE	31
7	**CHAPTER 7: CONCLUSIONS AND RECOMMENDATION**	**32**
	7.1 OUTLINE	32
	7.2 LIMITATIONS/CONSTRAINTS OF THE SYSTEM	32
	7.3 FUTURE ENHANCEMENTS	33
	7.4 INFERENCE	35
	MEMBER-WISE CONTRIBUTIONS	**36**
	APPENDIX A: SCREENSHOTS	**37**
	APPENDIX B: CODING	**44**
	REFERENCES	**49**

CHAPTER 1
PROJECT DESCRIPTION AND OUTLINE

1.1 INTRODUCTION

Reinforcement learning (RL) is a form of machine learning in which an *agent* learns by trial and error: it observes its environment, takes an action, receives a numerical *reward*, and gradually adjusts its *policy* (the rule that maps observations to actions) so that it collects more reward. In *continual* reinforcement learning the agent meets a sequence of tasks, one after another, and is expected to keep the skills it acquired on earlier tasks while it learns new ones [1].

With plain gradient-based training this rarely works. The network's parameters are shared by all tasks, so the updates that teach the new task also overwrite what the earlier tasks needed, and performance on old tasks collapses. This failure is called *catastrophic forgetting*. It is usually described as a trade-off between stability (keeping old knowledge) and plasticity (acquiring new knowledge) [1], [2]. The standard remedies fall into three families: regularisation, which penalises changes to parameters that mattered for earlier tasks [2]; replay, which mixes stored experience from earlier tasks into training [3]; and capacity isolation, which gives each task its own parameters [4].

A second family of remedies is inspired by the brain. Spiking neural networks (SNNs), in which artificial neurons communicate through discrete spikes instead of continuous values, promise event-driven, low-energy computation [5], [6]. Local learning rules such as spike-timing-dependent plasticity (STDP), which strengthens or weakens a connection depending on the relative timing of the spikes on either side of it, are proposed as biologically plausible complements to backpropagation [7], [8]. A sleep-like offline phase has been reported to recover forgotten tasks [9], and homeostatic plasticity, the brain's way of keeping firing rates in a working range, is often added to stabilise such systems [10].

This report describes NeuroPlast, a self-consolidating RL agent built as a test bed for these mechanisms. *Self-consolidating* refers to its sleep-like phase: at regular intervals the agent stops interacting with its environment and distils its own earlier behaviour, stored as states from previous tasks, back into its shared network. The agent works on MiniGrid grid-world tasks [11] and is trained with the PPO algorithm [12]. Each mechanism (the spiking encoder, STDP, homeostasis, a backprop-free learning rule, the sleep phase, a shared network and Transformer working memory) can be switched on or off independently, and

each was compared with a matched control that is not brain-inspired. The setting is *task-incremental*: the agent is told which task it is currently doing. The project does not assume that any mechanism helps; its purpose is to find out which ones contribute when the comparison is fair.

The rest of this chapter explains the motivation, states the problem and lists the objectives. Chapter 2 reviews related work, Chapter 3 lists the requirements, Chapter 4 presents the design, Chapter 5 the implementation, Chapter 6 the results and Chapter 7 the conclusions and future work.

1.2 MOTIVATION FOR THE WORK

Brain-inspired mechanisms for continual learning are popular, but they are usually introduced several at a time, in one system, and compared with an unmodified baseline. When such a system performs better, two questions remain open. First, which of the mechanisms did the work? Second, would a simpler non-biological control, given the same resources, have done just as well? Without answers to both, a reported improvement cannot be attributed to the brain-inspired idea itself.

Resources are a particular source of confusion. A sleep phase that replays stored experience may look better than ordinary replay simply because it replays more samples. A shared network may look more efficient than one network per task if only parameters are counted and the replay buffer it needs is ignored. A small number of random seeds can make a chance difference look significant. Each of these confounds affected our own early results, and correcting them changed the conclusions (Section 6.2.5).

The question also matters in practice. Agents that keep learning after deployment, such as robots or embedded devices, work under tight memory and compute budgets. If a brain-inspired mechanism helps, it should help per unit of memory and compute, not only in accuracy. Finally, negative results are useful: knowing which popular mechanisms do not help in a controlled setting saves effort for others, provided the test was fair.

1.3 PROBLEM STATEMENT

In a controlled task-incremental continual RL setting, which brain-inspired mechanisms improve learning or retention when each is isolated and compared with a matched control?

An improvement counts only if it exceeds the matched control by more than seed-to-seed variation, in a test whose comparison, statistic, seed count and stopping rule were fixed

before the runs. The controls are chosen to rule out the most obvious alternative explanation: a random update of the same size for STDP, replay with an exactly matched number of replayed samples for the sleep phase, and total-memory accounting (parameters plus replay buffer) for the comparison between a shared network and one network per task.

1.4 OBJECTIVE OF THE WORK

The objectives of the project were:

1. To test whether a **spiking encoder** reaches a better accuracy-versus-operations trade-off than a conventional convolutional encoder of the same topology that has been sparsified.
2. To test whether reward-modulated **STDP** improves learning beyond backpropagation alone, beyond homeostasis alone and beyond a random update of the same size.
3. To test whether **homeostasis** helps, with ordinary backpropagation and with direct feedback alignment.
4. To test **direct feedback alignment (DFA)**, a form of backprop-free credit assignment for the encoder, with and without homeostasis.
5. To test whether **sleep-like self-consolidation** reduces forgetting beyond experience replay and elastic weight consolidation, at a matched replay budget.
6. To test whether a **shared network trunk with per-task heads** retains more performance per unit of total memory than one network per task (parameter isolation).
7. To test whether **Transformer working memory** helps on a task that actually requires memory, compared with a memoryless agent and a stack of recent frames.
8. To keep every comparison **fair**: the same frames, seeds and learning rate in every arm, tests specified before the runs, and multiple-comparison correction.
9. To count **memory** as parameters plus everything stored, not parameters alone.
10. To make the work **reproducible**: every number in the results is regenerated by a script from the frozen run files, and the trained agents can be inspected through an offline dashboard.

1.5 SUMMARY

Continual RL suffers from catastrophic forgetting. Brain-inspired remedies are popular but are usually tested together and without matched controls, which leaves their individual contribution unknown. NeuroPlast isolates six brain-inspired mechanisms (the spiking encoder, STDP, homeostasis, DFA, sleep-like self-consolidation, and a shared trunk versus

isolation) and attempts a seventh (Transformer working memory), each against a matched non-biological control, with resource budgets matched and tests specified in advance. The next chapter places this work in the context of existing approaches.

CHAPTER 2
RELATED WORK INVESTIGATION

2.1 EXISTING APPROACHES/METHODS

This section groups the existing approaches into six families. The first three are the standard strategies against forgetting; the last three are the brain-inspired ideas that NeuroPlast tests.

Regularisation. Elastic weight consolidation (EWC) estimates how important each parameter was for earlier tasks, using the Fisher information, and penalises moving important parameters while a new task is learned [2]. It protects old tasks at the cost of plasticity for new ones. Learning without Forgetting (LwF) regularises the network's *outputs* instead of its parameters: before a new task it keeps a copy of the network and distils that copy's outputs on new-task inputs, so it needs no stored data [13].

Replay. CLEAR mixes new experience (for plasticity) with replayed experience from earlier tasks (for stability) and adds behavioural cloning on the replayed experience: a term that keeps the current policy close to the stored policy, and one that keeps the current value estimate close to the stored value [3]. It is simple, needs no task identities and is a very strong baseline. Its cost is the replay buffer, which grows with the number of tasks.

Capacity isolation. PackNet trains a task, prunes the least important weights, freezes the rest and uses the freed capacity for the next task, storing a mask per task [4]. The simplest form of isolation trains one separate network per task. Isolation cannot forget by construction, so the meaningful question is not whether a method forgets less than isolation, but whether it keeps high performance at lower total memory while exploiting transfer between tasks.

Sleep-like consolidation. Tadros et al. implement a sleep-like phase in feed-forward networks: offline, with noisy input and local unsupervised Hebbian plasticity, it recovers previously forgotten tasks in incremental classification [9]. Related work on self-consolidation distils accumulated experience into an agent's parameters instead of retrieving raw trajectories [14], which motivates counting the memory that replay needs.

Spiking neural networks and surrogate gradients. A spike is a step function of the neuron's membrane potential and has no useful derivative, so SNNs are trained end to end with *surrogate gradients*: a smooth stand-in derivative used only in the backward pass [15]. The slope of that surrogate matters much more in RL than in supervised learning

[16]. Toolkits such as SpikingJelly and snnTorch make SNNs easy to build [5], [17]. Spiking Transformer designs combine spiking computation with attention, for example for offline RL [18]; Spikingformer warns that many "spiking" architectures hide non-spike floating-point computation [6], which matters when operations are counted.

Local learning rules. STDP changes a synapse according to the timing of pre- and post-synaptic spikes [19]. Plain STDP has no notion of reward, so RL uses *three-factor* rules, in which a global signal such as reward or advantage modulates the local STDP term [7], [8], [20]. Direct feedback alignment (DFA) sends the output error to each hidden layer through a fixed random matrix, so a layer can learn without the exact transposed weights that backpropagation needs [21]. Homeostatic plasticity adjusts each neuron's excitability to keep its firing rate near a set point [10].

Evaluation. Continual World argues that forgetting alone is an incomplete metric and that forward transfer must also be measured [22]; forgetting itself is usually measured as in Chaudhry et al. [23]. Energy claims for SNNs usually rest on a proxy that charges an accumulate per spike-driven operation and a multiply–accumulate per dense operation, using 45 nm estimates [24].

2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS

Table 2.1 summarises the strengths and weaknesses of each family as they bear on this project.

Table 2.1: Pros and cons of existing approaches to forgetting

Approach	Pros	Cons
Regularisation (EWC; LwF)	No stored data (LwF) or a fixed-size penalty (EWC); simple to add	Protects old tasks at a cost in plasticity; EWC's cost grows with its strength λ
Replay (CLEAR)	Very strong in practice; simple; no task identities needed	Needs a replay buffer that grows with the number of tasks; its budget is rarely reported
Capacity isolation (one network per task; PackNet)	Cannot forget by construction	Parameters grow with the number of tasks; no transfer between separate networks
Sleep-like consolidation	Offline, needs no environment interaction; biologically motivated	Usually still relies on stored data; its replay budget is seldom matched to replay's
Spiking networks	Event-driven; fewer operations possible on neuromorphic hardware	Slower to simulate on CPUs and GPUs; energy savings are estimates without such hardware
Local rules (STDP, three-factor, DFA, homeostasis)	Biologically plausible; DFA avoids weight transport	Credit assignment is weak; unstabilised STDP can run away; often tested without matched controls

The gap. The brain-inspired mechanisms are typically introduced together and compared

against an unmodified baseline, without matching the resources that the strongest non-biological controls would use: the number of replayed samples, the total memory including any buffer, the number of seeds, and, for local rules, the size of the update. NeuroPlast addresses this gap by testing each mechanism alone against such a control, with the budgets stated and matched, and by reporting the results whether they are positive or negative.

CHAPTER 3
REQUIREMENT ARTIFACTS

3.1 INTRODUCTION

This chapter lists what the project needed in order to answer its research question: the hardware and software, the data (which in RL is generated rather than downloaded), the functional requirements of the software, its performance and security requirements, and the look and feel of the figures and the dashboard. Because the question is empirical, the most important requirement is that every comparison be fair and every number be reproducible.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

All experiments ran on CPUs; no GPU was used at any stage. The models are small, and most of the time in each run goes to stepping the MiniGrid environments, so the work was parallelised across CPU cores, one thread per run. Table 3.1 lists the hardware.

Table 3.1: Hardware used

Item	Specification and use
Experiment machine (cloud container)	4 logical CPUs (Intel(R) Xeon(R) Processor @ 2.80GHz), about 15 GB of RAM, no GPU. Ran the experiment queue: up to four runs at a time, one thread each (OMP_NUM_THREADS=1)
Development laptop	Intel(R) Core(TM) i3-7020U @ 2.30 GHz, 2 cores, 4 logical processors; 12 GB (11.9 GB usable) of RAM; SSD; Intel HD Graphics 620 (integrated), not used for training; Windows 10. Used for development, the dashboard and the live demo
Total compute (frozen study)	About 110 single-thread CPU hours for 289 PPO runs

Table 3.2 lists the software. `requirements.txt` states minimum versions only, so the table gives both the requirement and the version installed in the environment that regenerated this report.

Table 3.2: Software stack: the requirement in requirements.txt and the installed version

Package	requirements.txt	Installed version
Python	–	3.11.15
torch	torch>=2.2	2.14.0
numpy	numpy	2.4.6
gymnasium	gymnasium>=1.0	1.3.0
minigrid	minigrid>=3.0	3.1.0
snntorch	snntorch	1.0.0
scipy	scipy	1.17.1
pandas	pandas	3.0.6
matplotlib	matplotlib	3.11.2
PyYAML	pyyaml	6.0.1
pytest	pytest	9.1.1
LaTeX (paper), LibreOffice (this report), python-pptx (deck), Playwright (screenshots)	not in requirements.txt	tools for the deliverables only

The spiking neurons are implemented in the project itself rather than taken from a library, so that the per-timestep spikes needed by the STDP rule are directly available; snnTorch is used only as a cross-check in the unit tests [17]. The environments come from Gymnasium and MiniGrid [11], and PPO is a single-file implementation in the style of CleanRL [25].

3.3 SPECIFIC PROJECT REQUIREMENTS

3.3.1 DATA REQUIREMENTS

The project uses no fixed dataset. In RL the data are the agent's own experience, generated by interacting with an environment, so the data requirement is a set of environments with known properties. All are MiniGrid grid-worlds with the standard egocentric 7×7 view, encoded as 20 one-hot channels (object type, colour and state) [11]. Table 3.3 lists them.

Table 3.3: Environments and task sequences

Environment	What the agent must do	Used for	Frames
fetch3	A 6×6 room with three objects; task k rewards picking up object k; any other pick-up ends the episode with reward 0	Continual learning (3 tasks)	150k per task (400k for the SNN+Transformer agent)
fetch5	As fetch3, with five objects	Continual learning (5 tasks)	150k per task (450k for the fair 5-task SNN runs)
DoorKey-6x6	Pick up a key, open a door, reach the goal; a run either solves it or stays near zero	Local plasticity, homeostasis and DFA	400k
Memory S7, S11, S13	See a cue object at one end of a corridor and choose the matching object at the other end	The Transformer working-memory question	1M (S7 pilot); 2M–6M (S11/S13 probes)

In fetch3 and fetch5 the observations are statistically identical across tasks, so the same state demands different actions in different tasks. This guarantees interference under naive training, while navigation and picking up objects are shared skills that allow transfer. The agent is told the task identity; without it the tasks would be unsolvable, because the goal is not visible. The other data the project produces are stored run files: for every run, its configuration, training curve, evaluation results and final checkpoint. Replay and sleep keep a buffer of stored states, 5,000 per task by default, at a measured cost of 188 bytes per state (632 bytes for the SNN+Transformer agent's four-frame windows).

3.3.2 FUNCTIONAL REQUIREMENTS

- Train a PPO agent on a single task or on a sequence of tasks, with a CNN or SNN encoder and optional Transformer working memory.
- Switch each mechanism on or off independently through configuration: STDP and its random-update control, homeostasis, DFA, the sleep phase, and the baselines (naive fine-tuning, EWC, replay, isolation).
- Evaluate after every task on every task, producing the accuracy matrix R and the continual-learning metrics.
- Count operations per frame for the encoders, and memory as parameters plus stored states.
- Run the statistical tests specified for each comparison, and regenerate every number in the paper, the deck, the dashboard and this report from the frozen run files.
- Show the results and the trained agents on an offline dashboard, and run one episode of a trained agent on a CPU.

3.3.3 PERFORMANCE AND SECURITY REQUIREMENTS

- **Resumable runs.** Experiments run from job lists through a queue that skips finished jobs and claims running ones with lock files, so an interrupted queue can simply be restarted.
- **Thread pinning.** Each run uses one CPU thread (OMP_NUM_THREADS=1), so parallel runs do not compete for cores and timings are comparable.
- **CPU budget.** The frozen study had to fit on a 4-core machine: about 110 single-thread CPU hours for 289 runs. Frame budgets were chosen so that each arm of a comparison received the same number of frames.
- **Checkpoints.** Every run stores its final checkpoint so that its agent can be re-evaluated or shown in the demo without retraining.
- **Security and privacy.** The project processes no personal data. The dashboard is a single offline HTML file that makes no network requests, and the demo runs locally on a CPU.

3.3.4 LOOK AND FEEL REQUIREMENTS

- Figures are drawn at their printed size with a consistent colour for each method across all figures, the deck and the dashboard (sleep blue, replay orange, isolation green, EWC yellow, naive pink).
- Every figure shows the spread across seeds (individual seeds or 95% confidence intervals), not only means.
- The dashboard works offline as one file, in light and dark themes and at phone width, offers a table view for every chart, and states every result in plain language next to its key number.

3.4 SUMMARY

The project needs only commodity CPU hardware and an open-source Python stack. Its data are generated by MiniGrid environments chosen for specific properties: interference between tasks, bimodal single-task outcomes, and a need for memory. Its main non-functional requirements are fairness (matched budgets) and reproducibility (resumable runs and scripted numbers).

CHAPTER 4
DESIGN METHODOLOGY AND ITS NOVELTY

4.1 METHODOLOGY AND GOAL

The methodology is an *ablation study*: one agent with swappable parts, in which each brain-inspired mechanism is switched on alone and compared with a matched control that is not brain-inspired. The control is chosen to remove the most obvious alternative explanation for any improvement. Table 4.1 lists each mechanism, its implementation and its control.

Table 4.1: Mechanisms, implementations and controls (each row is switched on and off independently)

Mechanism	Implementation	Control / comparator
Spiking encoder	LIF convolutional network, 4 timesteps per frame, surrogate gradients; membrane reset every environment step	Same-topology CNN; operations at matched accuracy
STDP	Reward-modulated three-factor STDP on encoder synapses, added to the backprop update, stabilised by mean-centring and homeostasis	Random update of the same size, plus homeostasis; homeostasis alone
Homeostasis	Per-neuron bias pushes the firing rate towards 0.15	Same agent without it
DFA	Backprop-free encoder credit assignment: encoder layers learn from the output error through fixed random feedback (heads use exact gradients)	DFA without homeostasis; backprop
Sleep (self-consolidation)	Offline phases: old-task distillation on stored states and current-task self-distillation	CLEAR-style replay with the replayed samples matched exactly; EWC; naive fine-tuning
Shared trunk	One trunk shared by all tasks, with per-task heads and task embeddings	One network per task (isolation), at equal total memory; after the freeze, isolation at quarter width
Transformer memory (inconclusive)	Transformer over the last K encoded frames	Memoryless agent; a stack of the last K frames

Fair budgets. Within a comparison, every arm used the same frames, learning rate and seeds; for the frozen runs the seeds were 1 to n in every arm, and for the post-freeze follow-up 201–210. The one exception is the unmatched SNN+Transformer sleep arm (seeds 1–6), shown next to replay on seeds 1–8; the budget-matched test uses seeds 1–8 in both arms.

Prospective specification. The work was done in four phases. The first two were exploratory. For the last two, the comparisons, test statistics, seed counts and stopping rules were recorded in the project's version-controlled experiment log before the runs. This

is a project record, not a public registry. We call these tests *prospectively specified*. After the results were frozen, one more follow-up was specified and run in the same way; it is labelled *post-freeze* wherever it appears.

Statistics and labels. Solve rates are compared with Fisher's exact test and the Newcombe interval for a difference of proportions [26]; continuous metrics with Welch's t-test and its 95% confidence interval (CI) [27]; and where several contrasts form one planned family, Holm's correction is applied [28]. With few seeds the intervals are wide, so a non-significant difference is read as "no large effect" only where the interval excludes large effects, and never as equivalence. Every result in Chapter 6 carries one of three labels: **[confirmatory]** for a prospectively specified test run as specified, **[exploratory]** for a comparison not specified in advance (its p-values are uncorrected and should be read as descriptive), and **[descriptive]** for numbers given without a test.

Novelty. The novelty of this project is not a new mechanism. Every mechanism tested comes from cited prior work (Chapter 2). The contribution is the controlled, resource-matched ablation itself and what it found: the mechanisms are isolated one at a time, each is given a control that matches the resource a sceptic would point to (update size, replayed samples, total memory, and later network width), the tests are fixed in advance, and every earlier positive finding that did not survive these controls is reported with its cause.

4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS

Encoder. The encoder turns the 7×7 view into a feature vector. The CNN version has three 2×2 convolutions with 16, 32 and 64 channels and a linear layer to 128 features. The SNN version has the same topology with leaky integrate-and-fire (LIF) neurons: each neuron accumulates input into a membrane potential that leaks (factor β, initially 0.9, learnable), emits a spike when it crosses a threshold, and is reset. Each frame is presented for 4 timesteps, and the membrane is reset at every environment step, so the SNN has no memory across frames. Training uses a fast-sigmoid surrogate gradient with slope 25 [15].

Transformer working memory. In the SNN+Transformer agent, each of the last K frames is encoded and a one-layer Transformer encoder [29] attends over them, reading out the newest position; padding before the start of an episode is masked. On the fetch tasks $K = 4$.

Shared trunk and actor–critic heads. Every non-isolation agent is a shared trunk with per-task heads. The trunk (the encoder) is shared by all tasks; for the CNN agent it has 142,832 parameters. Each task adds an actor–critic head (17,032 parameters: a policy and a

value network) and a 128-dimensional task embedding, 17,160 parameters per task, so the 3-task agent has 194,312 parameters. Isolation instead adds a full network of 159,864 parameters per task. The parameters that grow with the number of tasks are therefore the heads and embeddings and, for replay and sleep, the replay buffer.

Plasticity rules. After every PPO optimiser step, the spikes recorded in that minibatch produce a local STDP update for each encoder layer (eligibility traces with pre/post pairing), multiplied by a third factor, the normalised advantage of each sample. The update is normalised to unit root-mean-square (RMS) size and scaled by α times the learning rate ($\alpha = 0.03$ in the specified comparison). The *stabilised* version mean-centres pre- and post-synaptic spikes and adds homeostasis. The *random* control applies an update with the same per-layer RMS but a random direction. Homeostasis nudges each neuron's bias towards a target firing rate of 0.15 with rate 0.01. DFA replaces the backpropagated gradient in the encoder with the output error sent through fixed random matrices [21].

Sleep phase (self-consolidation) and replay baseline. Every 25 PPO updates, and once at the end of each task, the sleeping agent goes offline for 700 gradient steps on two losses: distillation towards the stored policy and value on states saved at the end of each earlier task, and self-distillation of the current task towards a frozen snapshot taken at sleep onset. The replay baseline adds the same distillation loss on one replayed minibatch to every PPO minibatch, as in CLEAR [3]. Both keep CLEAR's cloning losses but not its off-policy RL loss, because PPO is on-policy. With the number of replayed states matched, the comparison therefore isolates *when* replay happens (offline versus interleaved), plus the self-distillation term.

Baselines. The non-biological comparators follow the three standard strategies: regularisation (an online EWC that keeps one accumulated Fisher estimate and one anchor copy of the parameters, with $\lambda = 10, 100, 1000$), replay (above), and capacity isolation (one fresh network per task), with naive fine-tuning as the lower bound. The post-freeze follow-up adds LwF [13] with a copy of the network stored in 8-bit integers (int8) as the teacher, distilling 128 current-task states per PPO minibatch; it stores no data between tasks.

Evaluation and statistics pipeline. After training on task i, the agent is evaluated on every task j for 100 episodes, giving the accuracy matrix entry R(i, j). Final accuracy (ACC) is the mean of the last row; forgetting (FORGET) is the average drop from each old task's best value to its final value [23]; forward transfer (FWT) compares the area under each task's training curve with a from-scratch network [22]. Single-task runs report the solve

rate (final evaluation return above 0.5 over 200 episodes) and the AUC, the mean training return over the run, which measures learning speed. A numbers pipeline reads every frozen run file and computes every number in the paper, the deck, the dashboard and this report.

4.3 SOFTWARE ARCHITECTURAL DESIGNS

Fig 4.1 shows the agent's data flow: the environment's view goes through the encoder, the optional Transformer memory and the per-task actor–critic heads, which choose the action. Fig 4.2 shows the continual training loop of the self-consolidating variant: online learning (wake) and offline consolidation (sleep) alternate on the same shared trunk, linked by the replay buffer.

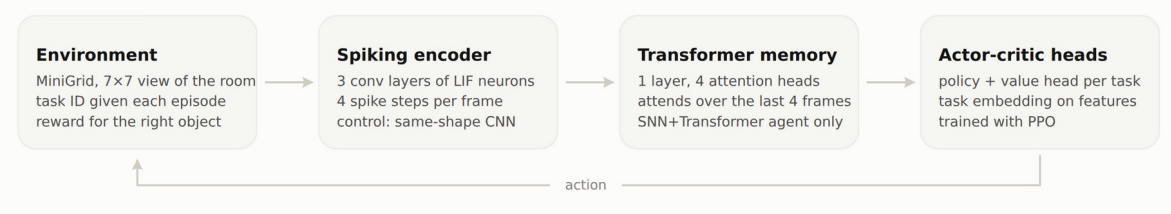


Fig 4.1: Architecture of the agent: environment, spiking encoder, Transformer working memory and actor–critic heads



Fig 4.2: Continual training of the self-consolidating variant: wake and sleep phases alternate on the same shared trunk, linked by the replay buffer

The code is organised so that each mechanism is one module and each experiment is one configuration file. Table 4.2 gives the layout of the repository.

Table 4.2: Repository layout

Path	Contents
neuroplast/models/	LIF neuron and surrogate gradient, CNN and SNN encoders with operation counting, Transformer memory, actor–critic heads, the agent
neuroplast/learning/	Trace STDP and eligibility, the hybrid STDP rule with its random control and homeostasis, DFA
neuroplast/sleep/, neuroplast/baselines/	Sleep phase; naive, EWC, replay, replay buffer
neuroplast/envs/, neuroplast/eval/	MiniGrid wrappers and task suites; continual-learning metrics
train.py, continual.py	Single-file PPO; the task-sequence runner (accuracy matrix, forgetting, transfer)
configs/, jobs/, scripts/	One YAML file per experiment; job lists; the resumable queue, statistics and memory accounting
runs/, results/	Frozen run files; aggregated tables
paper/, presentation/, demo/, report/	Paper and numbers pipeline, deck, dashboard and demo, this report
explore/	Post-freeze follow-up: its experiment log, runner, runs and tests

4.4 USER INTERFACE DESIGNS

The project's user interface is an offline results dashboard: one self-contained HTML file built from the frozen results by the same numbers pipeline as the paper. It opens with the headline answer and a tally of the four result groups (Fig 4.3), then shows the architecture, a mechanism scorecard, the trained agents, forgetting curves for each task sequence (Fig 4.4), the memory trade-off with the post-freeze follow-up, and the next steps. Its design follows the look-and-feel requirements: each method has one colour everywhere, every chart has a hover tooltip and a table view, the layout adapts to phone width, and the page works in light and dark themes. More screenshots are in Appendix A.

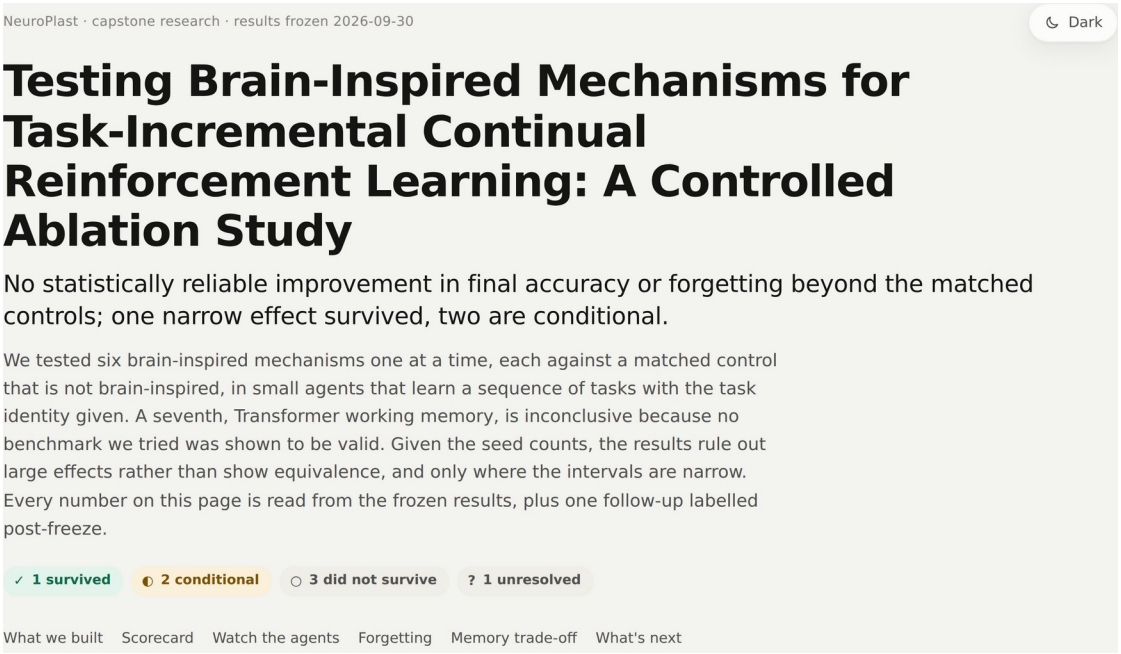


Fig 4.3: Dashboard: headline, plain-language summary and the tally of result groups (light theme)

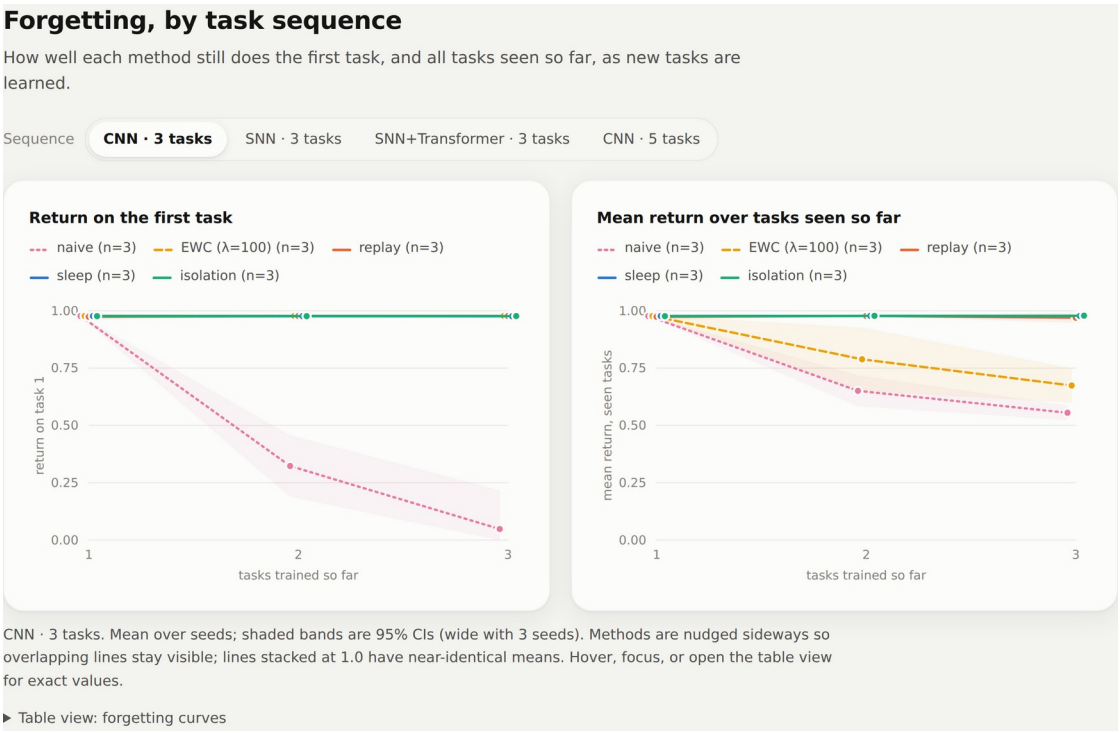


Fig 4.4: Dashboard: forgetting curves with a sequence selector; shaded bands are 95% confidence intervals

4.5 SUMMARY

The design is one agent with swappable modules and a matched control for every mechanism, evaluated with a fixed metric set and statistics specified in advance. The architecture separates the encoder, memory, heads, plasticity rules, sleep phase and baselines, so that each ablation changes exactly one component. The novelty lies in the controlled, resource-matched comparison, not in any individual mechanism.

CHAPTER 5
TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 OUTLINE

The implementation is written in Python with PyTorch and runs on CPUs. This chapter shows the key code solutions with short excerpts taken verbatim from the repository: the spiking neuron, the stabilised STDP rule and homeostasis, the matched random-update control, the sleep phase's replay budget, the memory accounting and the statistical tests. It then describes the experiment queue and the prototype (the dashboard and the demo script). Longer excerpts are in Appendix B. Table 5.1 lists the training protocol shared by all arms.

Table 5.1: Training protocol (identical in every arm of a comparison)

Setting	Value
PPO	16 parallel environments × 128 steps per rollout, 4 epochs, minibatch 256, clip 0.2, $\gamma = 0.99$, GAE $\lambda = 0.95$, early stop at approximate KL 0.03
Learning rate (Adam)	0.001 (CNN, SNN); 0.0003 (SNN+Transformer, memory probes)
Frames per task	150k (CNN, SNN); 400k (SNN+Transformer); 450k (5-task SNN, fair budget)
SNN	4 timesteps, learnable leak (initial 0.9), threshold 1, surrogate slope 25, membrane reset per frame
Plasticity	$\alpha = 0.03$ (specified comparison); homeostasis target 0.15, rate 0.01
Sleep	every 25 updates and at the end of each task; 700 steps per phase; 5,000 states per task
Evaluation	100 episodes per task (continual); 200 (single task); actions sampled from the policy

5.2 TECHNICAL CODING AND CODE SOLUTIONS

5.2.1 LIF NEURON AND SURROGATE GRADIENT

The spike is a step function, so its derivative is replaced in the backward pass by a fast-sigmoid surrogate. The neuron is stateless per call: the caller threads the membrane potential through the timesteps, which makes the per-timestep spikes available to the STDP rule. The reset term is detached from the gradient, as in snnTorch, for stable training.

Excerpt from neuroplast/models/snn/lif.py

```
class FastSigmoidSpike(torch.autograd.Function):
    @staticmethod
    def forward(ctx, v_minus_thr, slope):
        ctx.save_for_backward(v_minus_thr)
        ctx.slope = slope
        return (v_minus_thr > 0).to(v_minus_thr.dtype)
```

```
    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        grad = grad_out / (1.0 + ctx.slope * x.abs()) ** 2
        return grad, None

    def forward(self, current: torch.Tensor, v: torch.Tensor | None, s_prev:
        torch.Tensor | None):
        if v is None:
            v = torch.zeros_like(current)
            s_prev = torch.zeros_like(current)
        # reset term is detached (as in snnTorch's default) for stable gradients
        v = self.beta(current.ndim) * v + current - self.threshold *
            s_prev.detach()
        s = spike_fn(v - self.threshold, self.slope)
        return s, v
```

5.2.2 STABILISED STDP AND HOMEOSTASIS

Unstabilised STDP ran away in early experiments: because the frame is static across the SNN's timesteps, the same pre/post pairings repeat, and a mostly potentiating rule feeds back on itself. The stabilised rule mean-centres each neuron's spikes, which turns STDP into a covariance rule with no expected drift under independent firing, and homeostasis nudges each neuron's bias towards a target firing rate.

Excerpt from neuroplast/learning/hybrid.py

```
def _center(seq):
    """Subtract each neuron's (channel's) mean over batch, time [and space]:
    turns STDP into a covariance rule with zero expected drift for
    independent pre/post firing."""
    x = torch.stack(seq)  # (T, B, C[, H, W])
    dims = (0, 1) + tuple(range(3, x.dim()))
    x = x - x.mean(dim=dims, keepdim=True)
    return list(x)

    def _homeostasis(self, rec):
        """Local intrinsic plasticity: nudge each neuron's bias toward a target
            rate."""
        for i in self.idx:
            post = torch.stack(rec[i][1])  # (T, B, C[, H, W])
            dims = (0, 1) + tuple(range(3, post.dim()))
            rate = post.mean(dim=dims)
            self.enc.layers[i].bias += self.homeo_lr * (self.homeo - rate)
```

5.2.3 THE MATCHED RANDOM-UPDATE CONTROL

The control that separates “STDP helps” from “any perturbation of this size helps” is implemented in the same loop as STDP. Whatever the update's direction (STDP or random), it is rescaled to the same per-layer RMS, α times the learning rate, before it is added to the weights.

Excerpt from neuroplast/learning/hybrid.py (HybridSTDP.after_step)

```
            if self.mode == "random":
                dW = torch.randn_like(layer.weight)
            elif i < 3:
                dW = stdp_conv(pre, post, layer.kernel_size, mod, **self.kw)
            else:
                dW = stdp_dense(pre, post, mod, **self.kw)
            rms = dW.pow(2).mean().sqrt()
```

```
            self.last_rms[i] = rms.item()
            if rms < 1e-12:
                continue
            if self.mode == "hebbian":
                w = layer.weight
                norms = w.flatten(1).norm(dim=1)
                w += (self.alpha * self.lr / rms) * dW
                scale = norms / w.flatten(1).norm(dim=1).clamp_min(1e-12)
                w *= scale.view(-1, *([1] * (w.dim() - 1)))
            else:
                layer.weight += (self.alpha * self.lr / rms) * dW
```

5.2.4 SLEEP PHASE WITH A MATCHED REPLAY BUDGET

To compare sleep with replay at an exactly matched budget, a sleep run can be given a total replay budget equal to the measured number of replayed states of the replay run with the same seed. The budget is spread as evenly as possible over the sleep phases the schedule will run.

Excerpt from neuroplast/sleep/sleep.py (Sleep.plan_steps)

```
    def plan_steps(self, cfg):
        """Per-phase step counts summing to replay_budget / batch. Phases run only
            once the buffer is
        non-empty (tasks 1..T-1): every `period` updates plus one at the end of
            each such task."""
        assert self.replay_budget % self.batch == 0, "replay_budget must be a
            multiple of batch"
        n_steps = self.replay_budget // self.batch
        updates = max(cfg.total_frames // (cfg.num_envs * cfg.num_steps), 1)
        per_task = updates // self.period + (1 if self.end_of_task_sleep else 0)
        n_phases = per_task * (cfg.n_tasks - 1)
        q, r = divmod(n_steps, n_phases)
        return [q + (1 if i < r else 0) for i in range(n_phases)]
```

5.2.5 MEMORY ACCOUNTING

Total memory is fp32 parameters plus the replay buffer at its measured storage cost: 188 bytes per stored state for the CNN and SNN agents (a 147-byte uint8 observation, a mask, a task identifier, the teacher's logits and value) and 632 bytes for the four-frame windows of the SNN+Transformer agent.

Excerpt from scripts/memory_fair.py

```
OBS_B, FIXED_B = 147, 40          # per frame; mask/window adds K bytes
RESERVOIR = 5000                  # sleep's current-task reservoir
    (neuroplast/sleep/sleep.py)


def bytes_per_state(window):
    return OBS_B * window + window + FIXED_B
```

5.2.6 STATISTICAL TESTS

The numbers pipeline computes the difference of means with a Welch confidence interval, and the difference of solve rates with a Newcombe (Wilson score) interval and Fisher's exact test.

Excerpt from paper/compute_numbers.py

```
def welch(a, b):
    a, b = np.asarray(a, float), np.asarray(b, float)
    d = a.mean() - b.mean()
    va, vb = a.var(ddof=1) / len(a), b.var(ddof=1) / len(b)
    se = math.sqrt(va + vb)
    if se == 0:
        return d, d, d, 0.0
    df = (va + vb) ** 2 / (va ** 2 / (len(a) - 1) + vb ** 2 / (len(b) - 1))
    t = stats.t.ppf(0.975, df)
    return d, d - t * se, d + t * se, stats.ttest_ind(a, b, equal_var=False).pvalue


def wilson(k, n, z=1.959964):
    p = k / n
    den = 1 + z * z / n
    c = (p + z * z / (2 * n)) / den
    h = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / den
    return c - h, c + h


def newcombe(k1, n1, k2, n2):
    p1, p2 = k1 / n1, k2 / n2
    l1, u1 = wilson(k1, n1)
    l2, u2 = wilson(k2, n2)
    d = p1 - p2
    return d, d - math.sqrt((p1 - l1) ** 2 + (u2 - p2) ** 2), d + math.sqrt((u1 -
        p1) ** 2 + (p2 - l2) ** 2)
```

5.2.7 EXPERIMENT QUEUE AND REPRODUCIBILITY PIPELINE

Experiments are listed in job files, one job per line with a “done” marker. The queue skips finished jobs and claims running ones with a lock file holding the worker's process id, so an interrupted queue resumes where it stopped, and stale locks left by dead workers are taken over. After the runs, one command per deliverable regenerates everything from the frozen files: `python paper/compute_numbers.py` writes every number with its source, `python paper/figures/make_figures.py` the figures, `python demo/build_dashboard.py` the dashboard, `python presentation/build_deck.py` the deck and `python report/build_report.py` this report.

Excerpt from scripts/run_queue.py

```
def claim(marker):
    """Atomically claim a job via <marker>.lock holding our pid; stale locks (dead
        pid) are taken over."""
    os.makedirs(os.path.dirname(marker) or ".", exist_ok=True)
    lock = marker + ".lock"
    for _ in range(2):
        try:
            fd = os.open(lock, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
            os.write(fd, str(os.getpid()).encode())
            os.close(fd)
            return lock
        except FileExistsError:
            try:
                pid = int(open(lock).read().strip() or 0)
                os.kill(pid, 0)
                return None  # held by a live worker
            except (ProcessLookupError, ValueError):
                os.remove(lock)  # stale
            except PermissionError:
                return None
    return None
```

5.3 PROTOTYPE SUBMISSION

The code, the frozen run files, the dashboard and the demo are in the project's public repository, https://github.com/VT69/NeuroPlast. The prototype consists of two parts that run on any CPU, with no GPU and no network connection after installation:

Running the prototype

```
git clone https://github.com/VT69/NeuroPlast
cd NeuroPlast
pip install -r requirements.txt
python demo/build_dashboard.py      # writes demo/index.html; open it in any
    browser
python demo/live_episode.py --agent sleep --task 0
python demo/live_episode.py --agent naive --task 0 --window
```

Results dashboard. `python demo/build_dashboard.py` builds `demo/index.html`, a single offline HTML file that can be opened in any browser (a built copy is in the repository). It shows the scorecard, the two demo agents (naive fine-tuning and sleep) with recorded episodes and their real accuracy matrices, the forgetting curves and the memory trade-off (screenshots in Appendix A).

Live episode. `python demo/live_episode.py --agent sleep --task 0` runs one episode of a trained agent on the CPU and prints each step and the agent's accuracy matrix; `--agent naive` shows the naive agent, and `--window` opens a game window. The agents are the fetch3 CNN agents of seed 1 after the full three-task sequence, the same checkpoints as the dashboard. One episode is an anecdote; the measured score is the accuracy matrix (final rows 0.126, 0.542, 0.956 for the naive agent and 0.978, 0.979, 0.979 for the sleep agent).

Paper and deck. The paper is built with LaTeX from the numbers pipeline, and the slide deck is generated by a script from the same numbers.

5.4 SUMMARY

The implementation keeps each mechanism small and inspectable: a 40-line LIF neuron, an STDP rule that shares its scaling with its random control, a sleep phase whose replay budget can be matched exactly, a measured memory cost per stored state, and statistical tests in one script. The resumable queue and the numbers pipeline make the whole study reproducible from its frozen run files.

CHAPTER 6

PROJECT OUTCOME AND APPLICABILITY

6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

The system was used to run 289 PPO runs, about 110 single-thread CPU hours in total, with 3–10 seeds per arm in the main comparisons. After the results were frozen, one further prospectively specified comparison was run (the post-freeze follow-up, Section 6.2.2). Table 6.1 summarises the outcome for each mechanism in the four groups used throughout this chapter: what survived the controls, what is conditional, what did not survive, and what remains unresolved.

Table 6.1: Outcome by mechanism

Mechanism	Group	Key result (95% CI)	Label
Homeostasis with DFA (backprop-free encoder)	Survived	Learning speed (AUC) +0.326 [+0.101, +0.551], Holm p = 0.017; solve rate not significant	Confirmatory
Shared trunk with per-task heads	Conditional	200 states/task: similar mean ACC to full-width isolation in 46% of the memory; quarter-width isolation beat it after the freeze (0.972 vs 0.944)	Confirmatory; post-freeze
Spiking encoder	Conditional	Works at 16.6k operations per frame where the CNN needs 33k; measured CPU cost 5.2×	Descriptive
STDP (tested rule)	Did not survive	vs same-size random update: AUC +0.033 [−0.260, +0.326]	Confirmatory
Homeostasis with backprop	Did not survive	AUC +0.200 [+0.024, +0.377], Holm p = 0.117	Confirmatory
Sleep-like self-consolidation	Did not survive	vs replay at a matched budget: ACC +0.017 [−0.026, +0.060]	Confirmatory
Transformer working memory	Unresolved	Benchmark validity not established	Confirmatory (decision rule)

6.2 SIGNIFICANT PROJECT OUTCOMES

Headline. No mechanism produced a statistically reliable improvement in final accuracy or forgetting beyond its matched control. Given the seed counts, these results rule out large effects rather than show equivalence, and they rule out large effects only where the intervals are narrow. The subsections below give the evidence for each group.

6.2.1 SURVIVED: HOMEOSTASIS SPEEDS UP BACKPROP-FREE LEARNING

[confirmatory] With DFA as backprop-free encoder credit assignment (heads use exact gradients; 10 seeds per arm, the same seeds in both arms), homeostasis raised the AUC from 0.076 ± 0.101 to 0.402 ± 0.307 (difference +0.326 [+0.101, +0.551], 5.3×; Welch p = 0.009, Holm p = 0.017, significant). It also raised the solve rate from 2/10 to 7/10

(difference +0.50 [+0.07, +0.74]; Fisher p = 0.070, Holm 0.070), which is not significant, and the remaining seeds still failed outright (Fig 6.1). In our interpretation, homeostasis is a real but partial fix for DFA: it speeds up learning without weight transport in the encoder, but it does not make DFA reliable, and backprop with homeostasis (10/10 solved) remains better.

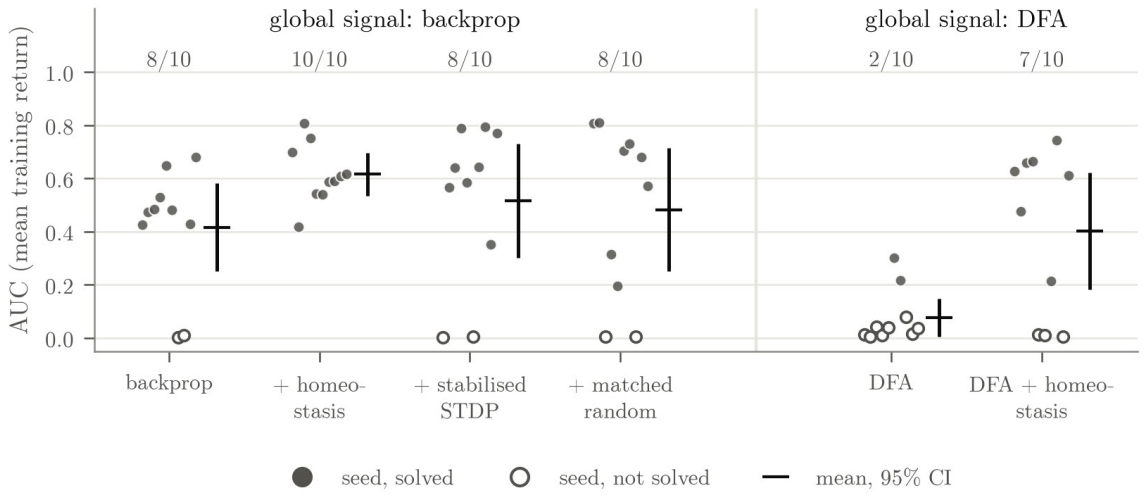


Fig 6.1: Local-plasticity arms on DoorKey-6x6 (10 seeds per arm). Dots are seeds' AUC (filled: solved); bars are means with 95% confidence intervals; numbers above the dots are seeds solved. Left: backprop as the learning signal; right: DFA with and without homeostasis

6.2.2 CONDITIONAL: SHARED TRUNK AND SPIKING ENCODER

Shared trunk versus isolation. [descriptive] Per parameter, the shared trunk with sleep is clearly more efficient than isolation (ACC per parameter 2.5–3.3 times higher). But at the default 5,000 stored states per task, the buffer outweighs the extra networks: isolation uses less total memory and gets 1.8–2.9 times more accuracy per MB on every sequence (Fig 6.2). [confirmatory] With a small buffer the picture changes. On the SNN, sleep with 200 states per task reaches a similar mean accuracy to isolation (0.838 ± 0.016 vs 0.833 ± 0.073; difference +0.005 [−0.167, +0.177], p = 0.921, 3 seeds per arm) in 0.89 MB against 1.92 MB, 46% of the memory. With so few seeds the interval is wide: it rules out neither a large deficit nor a large advantage, so this is not evidence of equal accuracy.

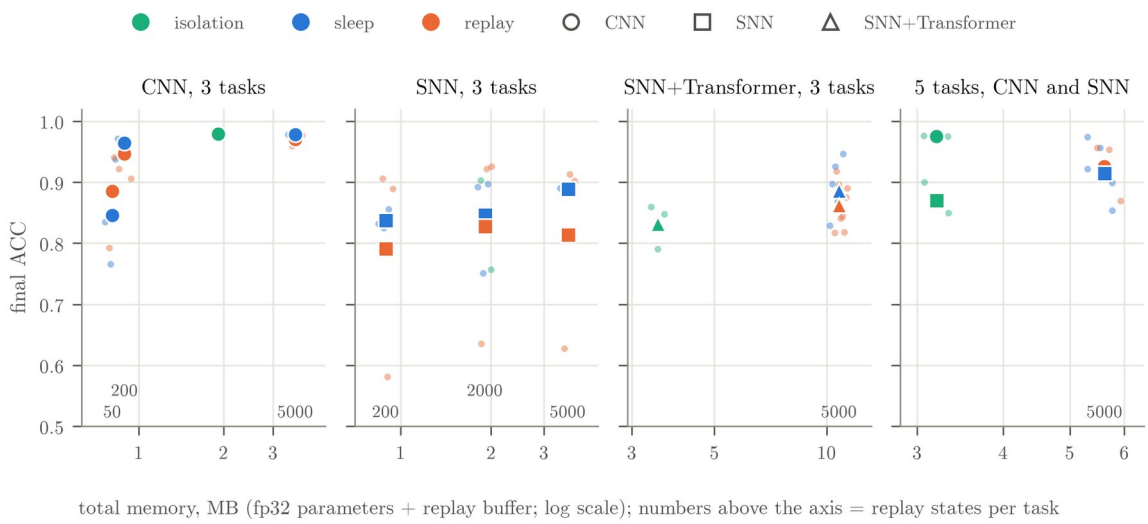


Fig 6.2: Final accuracy against total memory (fp32 parameters plus replay buffer, log scale). Numbers above the axis are replay states stored per task; the 5-task SNN runs use 450k frames per task

Post-freeze follow-up: isolation's network width. [confirmatory, post-freeze] The comparisons above use isolated networks of the same width as the shared trunk (159,864 parameters each). After the results were frozen, a follow-up specified in advance in its own experiment log tested narrower isolated networks on the 5-task CNN sequence, with seeds 201–210 in every arm (Fig 6.3, Table 6.2). Quarter-width isolation (44,928 parameters per task, 0.90 MB) reached 0.972 ± 0.002. A shared trunk trained with LwF and an int8 teacher (0.91 MB) reached 0.944 ± 0.018 (difference +0.028 [+0.015, +0.041], Holm p = 0.002), and the same trunk with replay (973 states per task, 1.83 MB) reached 0.920 ± 0.052 (difference +0.052 [+0.015, +0.089], Holm p = 0.011). Quarter width minus full width was −0.004 [−0.005, −0.002]. In our interpretation, the frozen isolation baseline's networks were much larger than these tasks need, so the small-buffer advantage of the shared trunk holds only against full-width isolation. Two limits apply: the shared trunk was not shrunk in the same way, and the SNN case is untested, because the only narrow SNN isolation runs were a two-seed pilot at 150k frames per task.



Fig 6.3: Post-freeze follow-up (CNN, 5 tasks, 10 seeds per arm): mean final accuracy on a 0.85–1.00 scale, with memory

Table 6.2: Post-freeze follow-up: isolation at the shared trunk's memory (CNN, 5 tasks, seeds 201–210)

Arm	Final ACC (mean ± SD)	Memory (MB)	Difference vs quarter-width isolation [95% CI]
Isolation, quarter width	0.972 ± 0.002	0.90	–
Shared trunk + LwF (int8 teacher)	0.944 ± 0.018	0.91	+0.028 [+0.015, +0.041] (Holm p = 0.002)
Shared trunk + replay	0.920 ± 0.052	1.83	+0.052 [+0.015, +0.089] (Holm p = 0.011)
Isolation, full width	0.975 ± 0.001	3.20	quarter − full: −0.004 [−0.005, −0.002]

Spiking encoder. [descriptive] Where both encoders work, they are equally accurate at matched operation counts (Fig 6.4). The sparsified CNN still works at 33k operations per frame (accuracy 0.998), but a stronger penalty silences it entirely. The SNN keeps working below that floor: with 2 timesteps it reaches 16.6k operations at accuracy 0.986, about 2.0 times fewer. Under the estimated arithmetic energy proxy this is 14.9 nJ per frame against 126 nJ for the sparsest working CNN; these are estimates that assume event-driven hardware, not measured power. The cost we did measure, in our CPU implementation, goes the other way: one PPO training step takes 80 ms for the SNN against 16 ms for the CNN (5.2×; CPU process time on one thread for one forward pass, backward pass and Adam update on 256 observations), because the SNN simulates several timesteps per frame.

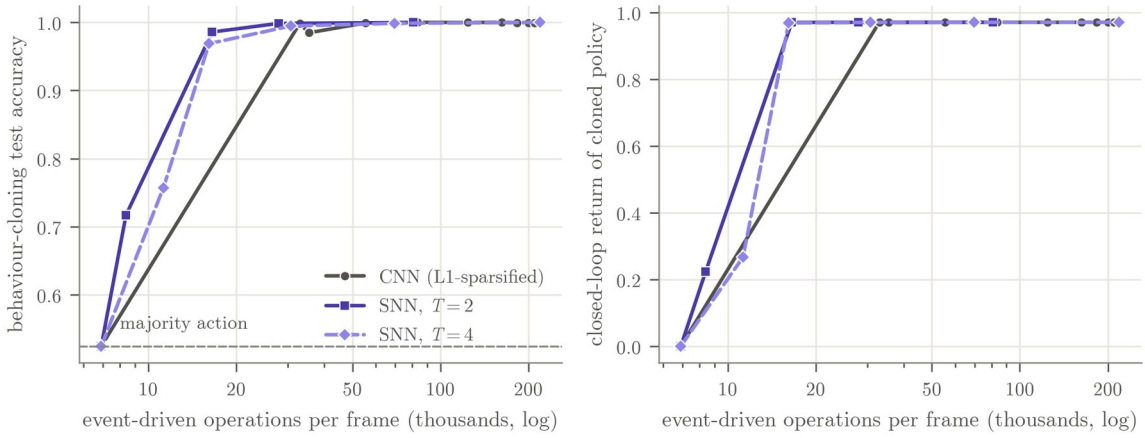


Fig 6.4: Accuracy against operation count for the spiking and the conventional encoder (behaviour cloning of a DoorKey-6x6 teacher, 2 seeds per point). Left: held-out action accuracy; right: closed-loop return of the cloned policy

6.2.3 DID NOT SURVIVE: STDP, HOMEOSTASIS WITH BACKPROP, SLEEP

STDP. [confirmatory] With backprop as the learning signal, the four arms solved DoorKey-6x6 at similar rates: backprop 8/10, with homeostasis 10/10, with stabilised STDP 8/10, and with the matched random update 8/10. All four specified contrasts are null

after Holm correction (Table 6.3). The tested rule (stabilised reward-modulated STDP at α = 0.03, added to the backprop update) was not distinguishable from a random update of the same size (AUC difference +0.033 [−0.260, +0.326], p = 0.818). This interval is wide, so it does not exclude a large effect in either direction: it means no detectable effect of this rule in this setup, not that STDP cannot help. [exploratory] Unstabilised STDP hurt: in an earlier DoorKey-5x5 experiment it solved 0/3 runs against 3/3 for backprop, with runaway firing (spike rate 0.39 against 0.20).

Table 6.3: Specified local-plasticity contrasts on DoorKey-6x6 (10 seeds per arm; first arm minus second; Holm-corrected p in brackets)

Contrast	Δ solve rate [95% CI]	Fisher p (Holm)	Δ AUC [95% CI]	Welch p (Holm)
STDP vs homeostasis alone	−0.20 [−0.51, +0.11]	0.474 (1.000)	−0.101 [−0.323, +0.121]	0.338 (0.724)
STDP vs matched random	+0.00 [−0.34, +0.34]	1.000 (1.000)	+0.033 [−0.260, +0.326]	0.818 (0.818)
Homeostasis vs backprop	+0.20 [−0.11, +0.51]	0.474 (1.000)	+0.200 [+0.024, +0.377]	0.029 (0.117)
Random vs homeostasis	−0.20 [−0.51, +0.11]	0.474 (1.000)	−0.134 [−0.372, +0.104]	0.241 (0.724)
DFA + homeostasis vs DFA	+0.50 [+0.07, +0.74]	0.070 (0.070)	+0.326 [+0.101, +0.551]	0.009 (0.017)

Homeostasis with backprop. [confirmatory] Homeostasis raised the AUC by +0.200 [+0.024, +0.377] (raw p = 0.029), which does not survive Holm correction (p = 0.117).

Sleep-like self-consolidation. [exploratory] Sleep and replay both remove most of naive fine-tuning's forgetting on every agent (Fig 6.5, Table 6.4). EWC protects old tasks at a plasticity cost that grows with λ: the return on the last task right after training it falls from 0.971 for naive fine-tuning to 0.603, 0.441 and 0.280 for λ = 10, 100 and 1000. At unmatched replay budgets, sleep never differs significantly from replay.

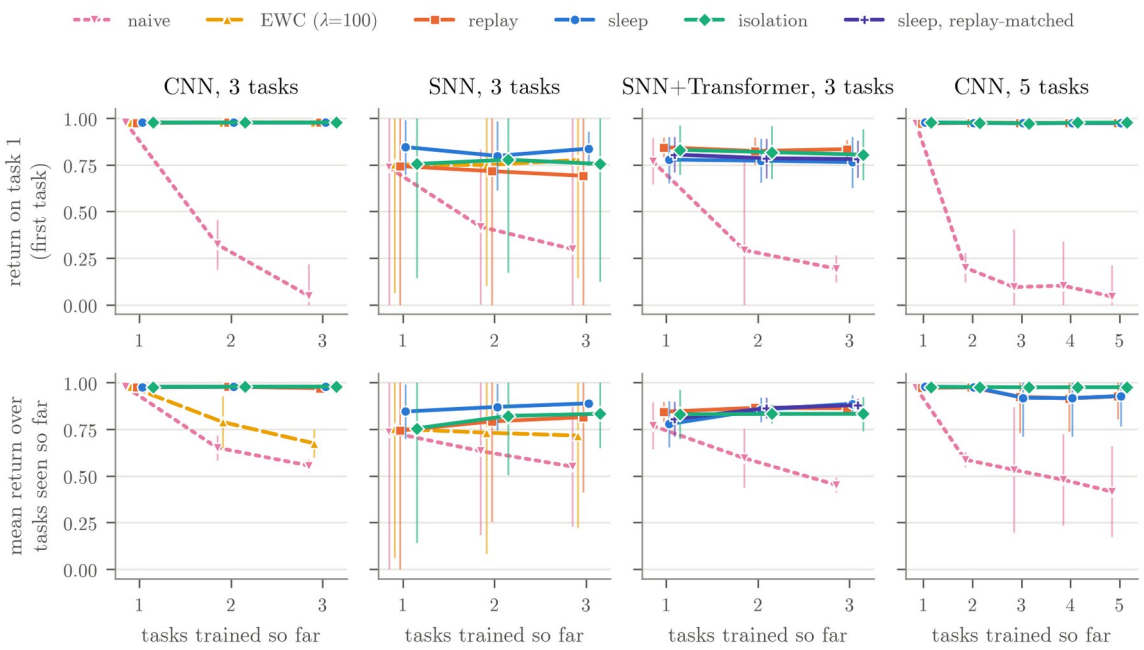


Fig 6.5: Forgetting on the continual task sequences. Top: return on the first task after each task is trained; bottom: mean return over the tasks seen so far. Means over seeds with 95% confidence intervals (clipped to [0, 1])

Table 6.4: Final ACC (mean ± SD over seeds) and FORGET on the continual sequences [exploratory; the matched row belongs to the confirmatory test]

	CNN, 3 tasks	**SNN, 3 tasks**	**SNN+Transformer, 3 tasks**	**CNN, 5 tasks**
Final ACC				
naive	0.555 ± 0.013	0.550 ± 0.130	0.452 ± 0.016	0.416 ± 0.098
EWC (λ = 100)	0.674 ± 0.030	0.716 ± 0.199	–	–
replay	0.970 ± 0.009	0.814 ± 0.162	0.862 ± 0.038	0.926 ± 0.049
sleep	0.978 ± 0.001	0.889 ± 0.003	0.887 ± 0.045	0.928 ± 0.065
sleep, matched	–	–	0.879 ± 0.043	–
isolation	0.979 ± 0.001	0.833 ± 0.073	0.832 ± 0.037	0.976 ± 0.000
FORGET (mean)				
naive	0.631	0.321	0.536	0.612
replay	0.002	0.023	0.019	0.006
sleep	0.001	0.006	0.006	0.003
sleep, matched	–	–	0.020	–
sleep − replay, ACC [95% CI]	+0.008 [−0.015, +0.030]	+0.074 [−0.328, +0.476]	+0.024 [−0.026, +0.075]	+0.002 [−0.133, +0.137]

[confirmatory] On the SNN+Transformer agent, the original sleep runs had consumed 28% more replayed states than replay. With an exactly matched budget, sleep was not distinguishable from replay on final accuracy (0.879 ± 0.043 vs 0.862 ± 0.038; difference +0.017 [−0.026, +0.060], p = 0.415, 8 seeds per arm, the same seeds in both arms), and unlike the STDP contrasts this interval is narrow (Fig 6.6). [descriptive] Forgetting was also close (0.020 ± 0.011 vs 0.019 ± 0.017). In our interpretation, sleep's small earlier edge

came from its extra replay, not from consolidating offline: in our implementation sleep is a way of scheduling replay, and at an equal budget we found no advantage for it. This is one sleep schedule on one agent; it does not show that offline consolidation can never help.

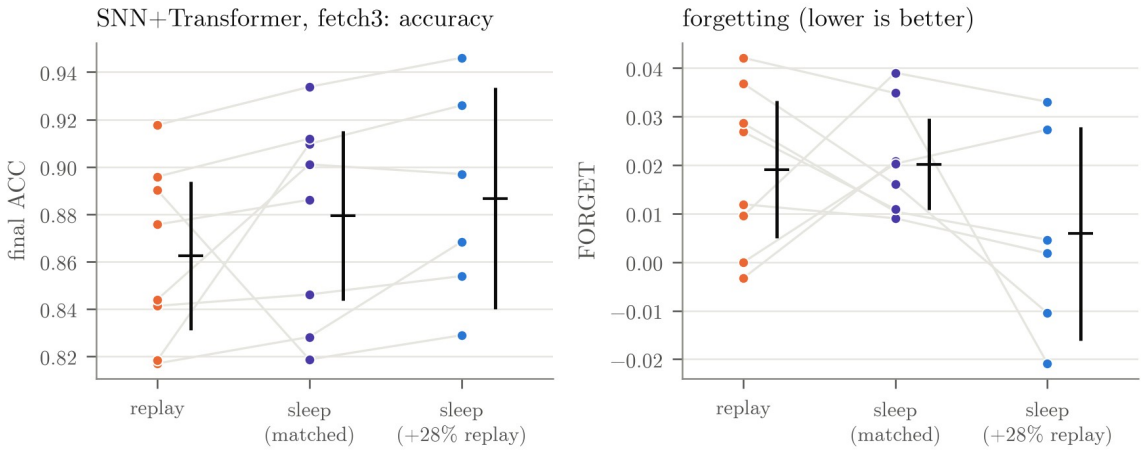


Fig 6.6: Sleep versus replay at an exactly matched replay budget (SNN+Transformer, fetch3; 8 seeds per arm). Grey lines connect runs with the same seed. Left: final accuracy; right: forgetting

6.2.4 UNRESOLVED: TRANSFORMER WORKING MEMORY

[exploratory] On the fetch tasks, which are fully determined by the current frame, the Transformer has nothing to remember. On MiniGrid-Memory S7, a memoryless CNN already succeeded in 0.915 of episodes, so S7 does not isolate memory. [confirmatory] On S11 and S13, where the cue is beyond the agent's view at the junction, the memoryless CNN was at chance, as required (0.430 and 0.485 success), but so was a CNN with a 12-frame stack (0.465 and 0.460) (Fig 6.7). Neither map met the criterion written before the probes, so the Transformer comparison was not run. [exploratory] A longer run of the stacked CNN on S11 for 6M frames also stayed at chance (0.485). No agent learned to go back and look at the cue; they walk to the junction and guess. The question is therefore inconclusive, not answered in either direction. The SNN version of the isolation-width comparison is likewise unresolved.

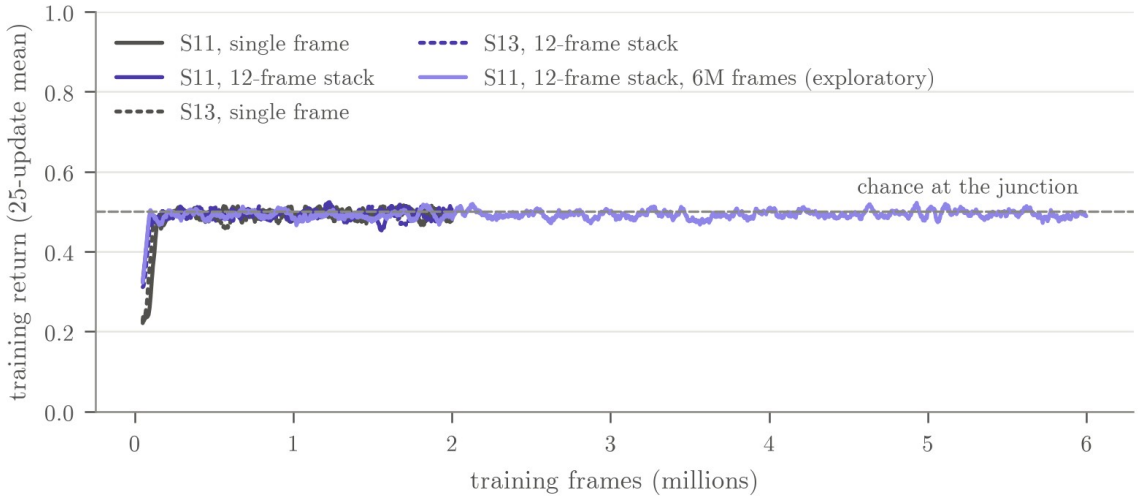


Fig 6.7: Memory-benchmark probes on MiniGrid-Memory S11 and S13 (CNN, one seed each): single frame versus a stack of the last 12 frames. A return of 0.5 is chance

6.2.5 WHAT DID NOT SURVIVE CONTROLS

Several results looked positive before the controls were in place. Table 6.5 lists them with the control that removed them; every "evidence then" entry is exploratory. The causes recur: too few seeds, too small a training budget for fresh networks, an unmatched replay budget, memory counted without the buffer, a benchmark that did not need memory, and, found after the freeze, a fixed isolation width.

Table 6.5: Earlier positive findings that shrank or disappeared once a control was added

Earlier finding	Evidence then	After the control	Cause	Label
Sleep beats replay (SNN+Transformer)	seeds 1–3: +0.039 [−0.006, +0.084], p = 0.072	seeds 1–6: p = 0.224; budget-matched, 8 seeds: +0.017 [−0.026, +0.060], p = 0.415	Seed count; replay budget	Conf.
Shared trunk far more efficient than isolation	ACC per parameter: 2.5–3.3×	Per total memory at the default buffer, isolation is 1.8–2.9× more efficient; 200 states/task (SNN): similar mean ACC in less than half the memory	Memory accounting	Desc.; conf. (SNN)
Sleep far beats isolation on 5 tasks (SNN)	150k/task, 2 seeds: 0.778 ± 0.007 vs 0.475 ± 0.023	450k/task, 3 seeds: +0.044 [−0.009, +0.098], p = 0.078	Training budget	Conf.
Homeostasis helps backprop	3 vs 5 seeds: 3/3 solved, p = 0.088	10 seeds: +0.200 [+0.024, +0.377], Holm p = 0.117	Seed count; multiple comparisons	Conf.
Homeostasis makes DFA work	3/3 vs 1/5 solved	10 seeds: 7/10 vs 2/10, Fisher p = 0.070 (AUC effect survives)	Seed count	Conf.
Transformer helps on a memory task	S7 pilot: 0.970 vs 0.465 (frame stack)	Memoryless CNN on S7: 0.915; on S11/S13 no memory agent learns	Benchmark validity; learning-rate confound	Conf. (rule)
With a small buffer, a shared trunk is more memory-efficient than isolation	Against full-width isolation: CNN, 200 states/task 1.08 vs 0.51 ACC per MB	Quarter-width isolation, CNN 5 tasks, 10 seeds: 0.972 at 0.90 MB vs LwF 0.944 at 0.91 MB (+0.028 [+0.015, +0.041])	Network width not matched	Conf., post-freeze

6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

The results apply most directly to agents that must keep learning after deployment under tight memory and compute budgets: mobile robots that meet new tasks in the field, embedded and edge devices that adapt on the device instead of in a data centre, and other systems where storing past experience is expensive. We did not build or test such systems; the points below state where the findings are relevant, not what we demonstrated.

- **Memory budgets decide the method.** Whether a shared network or separate per-task networks is cheaper depends on the replay buffer and on the size of each network. A deployed system should account for its total memory, buffer included, before choosing.
- **Energy claims need hardware.** Spiking encoders reach fewer operations per frame, but on ordinary CPUs they were slower to train. Any energy benefit depends on event-driven hardware and should be measured there.
- **Matched controls before adoption.** A mechanism that looks helpful should be compared with a simple control of the same cost (a random update of the same size, replay with the same number of samples) before it is adopted, as these controls removed most of our early effects.
- **Learning without backpropagation.** Homeostasis made backprop-free (DFA) training of a spiking encoder faster, which is relevant to on-chip learning where backpropagation is hard to implement, though it did not make it reliable.

6.4 INFERENCE

Across six tested mechanisms, none improved final accuracy or forgetting reliably beyond a matched non-biological control. One narrow effect survived (homeostasis speeding up DFA), two are conditional (the shared trunk's memory advantage, which depends on the buffer size and on isolation's network width, and the spiking encoder's lower operation count, which depends on hardware we did not use), and one question is unresolved (Transformer working memory). The agent's self-consolidation phase did no better than ordinary replay at an equal replay budget. The most consistent finding is methodological: most early positive effects disappeared once seeds, training budgets, replay budgets, memory accounting and network width were made fair.

CHAPTER 7
CONCLUSIONS AND RECOMMENDATION

7.1 OUTLINE

We isolated six brain-inspired mechanisms in a small task-incremental continual RL setting and gave each a matched control; a seventh, Transformer working memory, remained inconclusive because benchmark validity was not established. No mechanism produced a statistically reliable improvement in final accuracy or forgetting beyond its matched control; given the seed counts, these results rule out large effects rather than show equivalence, and only where the intervals are narrow.

- **Survived:** homeostasis sped up learning of a spiking encoder with backprop-free encoder credit assignment (DFA; heads use exact gradients), without making it reliable.
- **Conditional:** with a small replay buffer, a shared trunk with per-task heads and sleep reached similar mean accuracy to full-width isolation in less than half the memory, but after the freeze quarter-width isolation beat shared-trunk methods at equal or lower memory on a 5-task CNN sequence; and the spiking encoder reached a lower operation-count regime whose energy value is an estimate and whose measured CPU cost in our implementation was higher.
- **Did not survive:** the tested stabilised STDP rule against a same-size random update, homeostasis with backprop, and sleep-like self-consolidation against replay at a matched replay budget, together with the earlier positive findings of Section 6.2.5.
- **Unresolved:** the Transformer question and the SNN version of the isolation-width comparison.

The broader lesson is methodological: in this setting, the earlier positive effects of biological mechanisms shrank or disappeared once seed counts, training budgets, replay budgets, memory accounting and network width were made fair. We recommend that claims of this kind be reported with those budgets stated and matched.

7.2 LIMITATIONS/CONSTRAINTS OF THE SYSTEM

The limitations below are those stated in the project paper.

Task identity is given. Our setting is task-incremental: the agent knows which task it is doing. Removing the task identity from fetch3 would make it unsolvable (the goal is not

visible), so a task-agnostic study needs a different benchmark, and sleep's distillation, which relies on per-task heads, would need redesigning.

Scale. All agents are small (159,864–327,800 parameters), the environments are MiniGrid gridworlds, sequences have 3 or 5 tasks, and budgets are 150k–450k frames per task. Results may not transfer to larger agents, longer sequences or visually rich tasks.

CPU only, few seeds. Everything ran on a 4-core CPU. Many continual arms have only 3 seeds, so their CIs are wide and a tie means at most "no large difference" where the interval is narrow, never equivalence. The prospectively specified comparisons used 8–10 seeds per arm, and the post-freeze follow-up 10.

Energy is an estimated arithmetic energy proxy. The energy numbers multiply operation counts by literature constants for 45 nm arithmetic and assume event-driven hardware. We measured no power. The only cost we measured is CPU time in our implementation, and there the SNN was slower.

Single settings. Hyperparameters (homeostasis target, α, EWC λ beyond three values, sleep schedule) were not tuned per arm. A different homeostasis target or sleep schedule could change the results.

Baselines are simple. In the frozen study, isolation uses one full-width network per task; the post-freeze follow-up shows that narrower isolated networks change the memory comparison (Section 6.2.2), and PackNet-style masks would be cheaper still. The shared trunk was never shrunk in the same way. Our sleep phase is replay plus distillation, not the sleep replay consolidation of Tadros et al. [9].

The Transformer question is inconclusive. On S7 memory was unnecessary; on S11 and S13 no agent, including a CNN with a 12-frame stack, learned to use the cue, even at 6M frames. Benchmark validity was therefore not established, and the comparison was not run.

Prospective specification was informal. The test plans were recorded in the project's version-controlled experiment log before the runs, not in a public registry. The first two phases were exploratory, and their positive findings are exactly the ones Section 6.2.5 revisits. The follow-up was planned after the frozen results were known, although each of its tests was specified before its own runs.

7.3 FUTURE ENHANCEMENTS

Everything in this section is a plan or an open question, not a result; we make no prediction about what these studies will find.

Scaling up. The main limitation of this study is scale. Every experiment ran on a 4-core CPU, and the bottleneck was CPU-bound environment stepping, not model size: the agents are small, and most of each run's time went to simulating MiniGrid. This is what limited the seed counts, the sequence lengths and the training budgets. We plan to port the agent, the mechanisms and the baselines to JAX and run them on cloud GPUs with XLand-MiniGrid [30], a GPU-native implementation of MiniGrid that is reported to simulate millions of environment steps per second on one GPU, with a rule-and-goal system for generating many related tasks.

Validation first. Before trusting any new result, we will replicate the headline results on XLand-MiniGrid's ports of the MiniGrid environments: sleep versus replay at a matched replay budget, STDP versus a random update of the same size, and the post-freeze isolation-width result. Any result that does not replicate will be investigated before new comparisons are run.

Questions the scaled study will address.

- Longer task sequences (10–50 related tasks) where transfer matters, with full accuracy-versus-memory curves in which every method shrinks along its own size setting: isolation width, shared-trunk width and buffer size.
- 10–20 seeds per arm, with a power analysis before the runs and equivalence tests with stated margins, so that the current "no large effect" results become informative.
- A memory benchmark that agents can actually learn; then Transformer versus GRU versus frame stack, and a stateful SNN that carries its membrane state across frames.
- The SNN version of the isolation-width comparison at an adequate training budget (450k frames per task), and whether the conclusions change with network size.
- A task-agnostic setting with observable goals, so that the task identity can be removed.

Direction for publication: TinyML and embedded systems. Following our supervisor's direction, the next phase will target a TinyML / embedded-systems IEEE venue, where memory and compute are the binding constraints. This requires four additions to the study, all of them planned work:

1. **Full training-time memory accounting:** not only the memory kept after training (parameters and buffer) but also the peak memory during training (activations, optimiser state, snapshots and reservoirs).
2. **int8 versions of every method:** parameters, buffers and teacher networks stored in 8-bit integers, as the LwF teacher already was in the follow-up, so that methods are

compared at deployable precision.

3. **Accuracy-versus-memory curves:** for every method, swept along its own size setting, instead of single operating points.
4. **Measurements on real embedded hardware:** latency, memory and energy measured on a microcontroller or edge board, replacing the estimated arithmetic energy proxy.

Smaller follow-ups. A replay-budget sweep for sleep versus interleaved replay, motivated by an exploratory result in the follow-up log in which sleep forgot more than interleaved replay when replay was scarce; splitting sleep into offline replay alone versus offline replay plus self-distillation; plots of the STDP runaway mechanism; a stronger CNN sparsifier such as k-winners-take-all; compressed or generative replay to shrink the buffer [31]; a PackNet variant that can regrow capacity; and one standard benchmark, such as Continual World [22], for comparability. The methodology will stay the same: tests prospectively specified in a version-controlled experiment log, fresh seeds for confirmation, matched resources and every failure logged.

7.4 INFERENCE

NeuroPlast set out to find which brain-inspired mechanisms help continual RL when each is isolated and matched against a fair control. In a small task-incremental setting, the answer is that none produced a statistically reliable improvement in final accuracy or forgetting, one narrow effect on learning speed survived, and two advantages hold only under stated conditions. The agent's sleep-like self-consolidation worked as well as replay but, at an equal budget, no better. The project's main contribution is the controlled, resource-matched method and the record of which effects did not survive it. Scaling the same method up, and moving it towards memory-constrained embedded hardware, is the natural next step.

MEMBER-WISE CONTRIBUTIONS

The table below records each team member's contribution to the project.

Member	Registration no.	Contribution
Vaibhav Tiwari	23BAI10108	Project lead and principal contributor. Defined the research questions and designed the controlled ablation study; implemented the agent (spiking and convolutional encoders, Transformer working memory, plasticity rules, sleep phase and baselines), the experiment queue and the statistics and numbers pipeline; ran and analysed all experiments, including the post-freeze follow-up; built the results dashboard and the live demo; and prepared the paper, the presentation and this report.
Kavish Bishnoi	23BAI10789	[TBD: contribution, in the member's own words]
Kushal Upadhyay	23BAI10259	[TBD: contribution, in the member's own words]
Yash Yadav	23BAI10484	[TBD: contribution, in the member's own words]
Adamya Upadhyay	23BAI11221	[TBD: contribution, in the member's own words]
Siddharth Ubbott	23BAI10997	[TBD: contribution, in the member's own words]

APPENDIX A: SCREENSHOTS

The screenshots below show the offline results dashboard (`demo/index.html`, light theme), frames from the recorded episodes of the two demo agents, and their accuracy matrices.



Mechanism scorecard

The same conclusions as the paper. Brackets are 95% confidence intervals; p-values are two-sided. Evidence labels: *confirmatory* = a test prospectively specified in a version-controlled experiment log before its runs; *exploratory* = not specified in advance; *descriptive* = no test. All arms of each comparison used the same seeds.

Mechanism and question	Verdict	Key number	Evidence
Spiking encoder Does a spiking encoder reach the same accuracy with fewer operations than a sparsified ordinary network?	◐ Fewer operations in an estimate; higher measured CPU cost	Works at 16.6k operations per frame (accuracy 0.986); the sparsified CNN stops working below 33k. Measured cost in our CPU implementation: 80 vs 16 ms per training step (5.2×).	descriptive 2 seeds per point; energy is an estimated arithmetic energy proxy, not measured on hardware
Local learning rule (STDP) Does the tested rule (stabilised reward-modulated STDP, α = 0.03, added to backprop) help on DoorKey-6x6, compared with a random update of the same size?	○ Not distinguishable from a same-size random update	Difference in learning speed vs the random update: ΔAUC +0.033 [−0.260, +0.326], p = 0.818. The interval is too wide to exclude a large effect either way.	confirmatory 10 seeds per arm (same seeds), Holm-corrected
Homeostasis, backprop-free encoder Does homeostasis (keeping firing rates near a target) help an SNN whose encoder credit assignment is backprop-free (DFA; the heads use exact gradients)?	✓ Faster learning, not more reliable	Learning speed ΔAUC +0.326 [+0.101, +0.551] (Holm p = 0.017). Solved 7/10 vs 2/10 runs, not significant (Holm p = 0.070).	confirmatory 10 seeds per arm (same seeds)
Homeostasis, with backprop Does homeostasis help SNNs trained with ordinary backprop?	○ Not significant after correction	ΔAUC +0.200 [+0.024, +0.377]; raw p = 0.029, Holm-corrected p = 0.117.	confirmatory 10 seeds per arm (same seeds)
Sleep-like consolidation Does offline, sleep-like consolidation beat replay when both replay the same number of stored states?	○ No detectable difference from replay at a matched budget	Sleep minus replay, final accuracy: +0.017 [−0.026, +0.060], p = 0.415 (a narrow interval). Forgetting 0.020 ± 0.011 vs 0.019 ± 0.017 (descriptive).	confirmatory (accuracy; forgetting descriptive): replayed samples matched exactly, 8 seeds per arm (same seeds); both beat naive fine-tuning
Shared trunk with per-task heads Is one shared trunk with a head per task more memory-efficient than a separate network per task (isolation)?	◐ Only with small buffers and full-width isolation	SNN: sleep with 200 states per task reaches a similar mean accuracy to full-width isolation (0.838 vs 0.833; difference +0.005 [−0.167, +0.177]) in 0.89 vs 1.92 MB. Post-freeze (CNN, 5 tasks): quarter-width isolation 0.972 at 0.90 MB beat the shared trunk with LwF (0.944, 0.91 MB) and with replay (0.920, 1.83 MB).	confirmatory SNN 3 seeds per arm, wide CI; post-freeze 10 seeds per arm. The shared trunk was not shrunk the same way; the SNN case is untested
Transformer working memory Does Transformer working memory help on a task that needs memory?	? Inconclusive: benchmark validity not established	On MemoryS11/S13 a memoryless CNN (0.430, 0.485) and a 12-frame stack (0.465, 0.460) both stay at chance, so the comparison could not be run.	confirmatory (decision rule): the prospectively specified rule skipped the comparison

✓ survived ◐ conditional ○ did not survive ? unresolved

Fig A.1: Dashboard: mechanism scorecard with result groups, key numbers and evidence labels

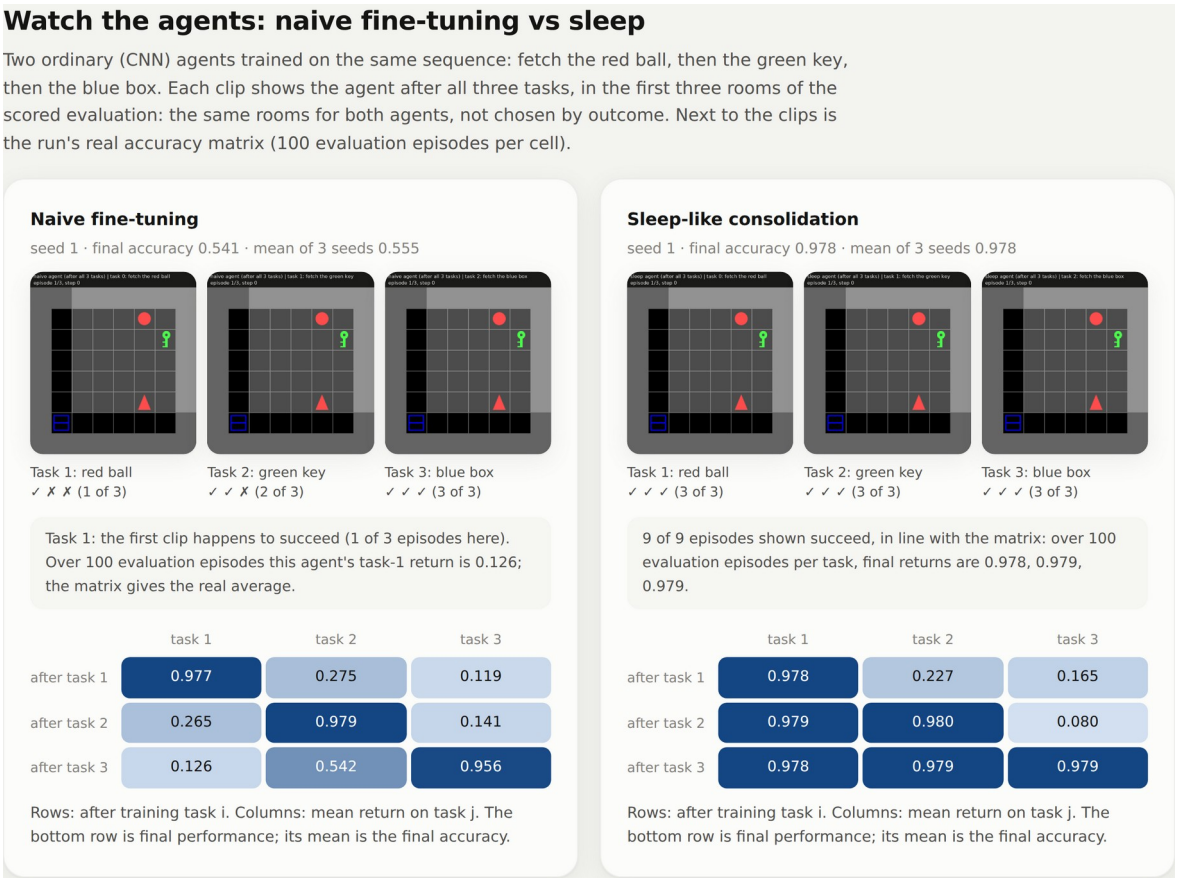


Fig A.2: Dashboard: the naive and sleep agents after the full fetch3 sequence, with recorded episodes and accuracy matrices

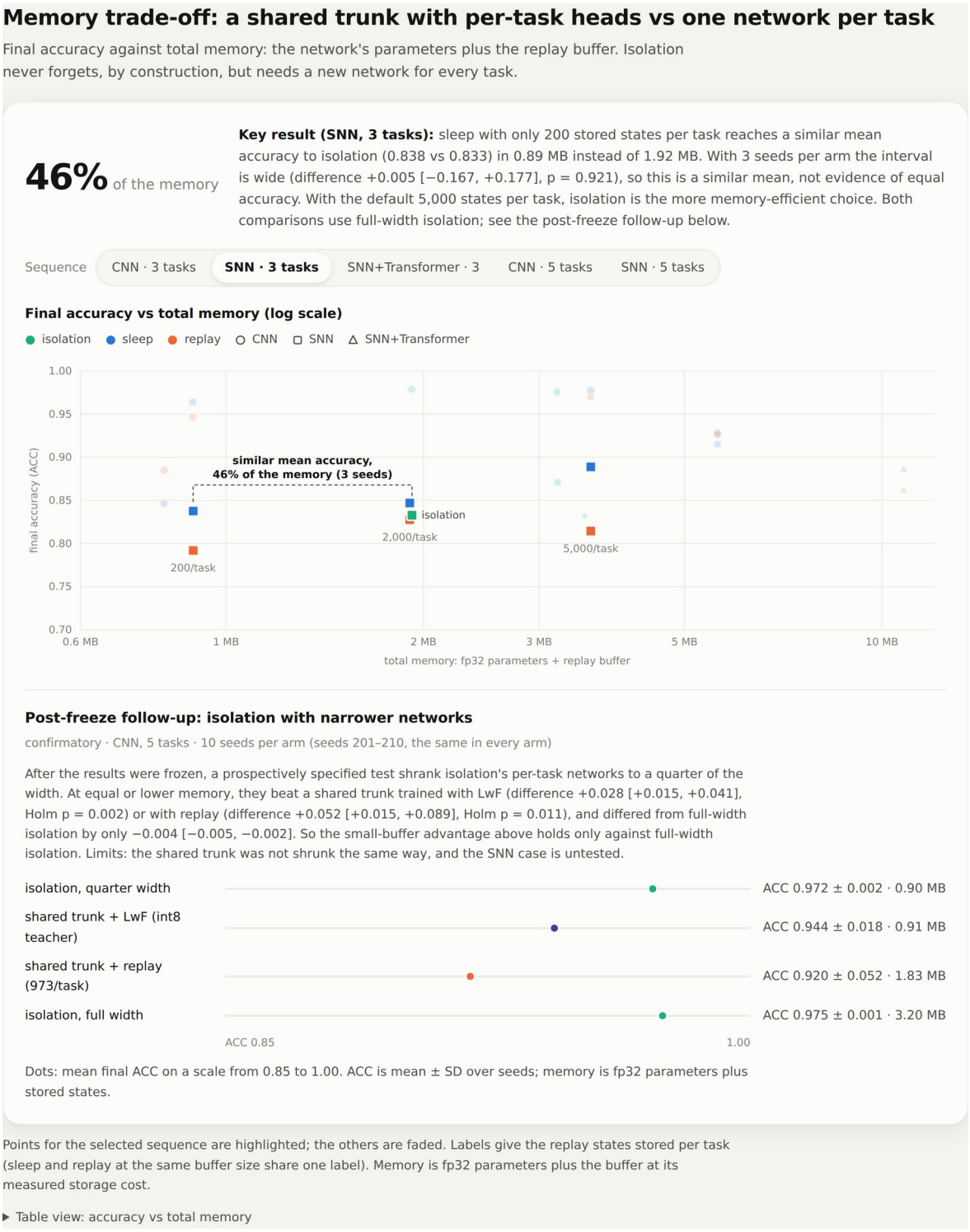


Fig A.3: Dashboard: memory trade-off, with the key result and the post-freeze follow-up

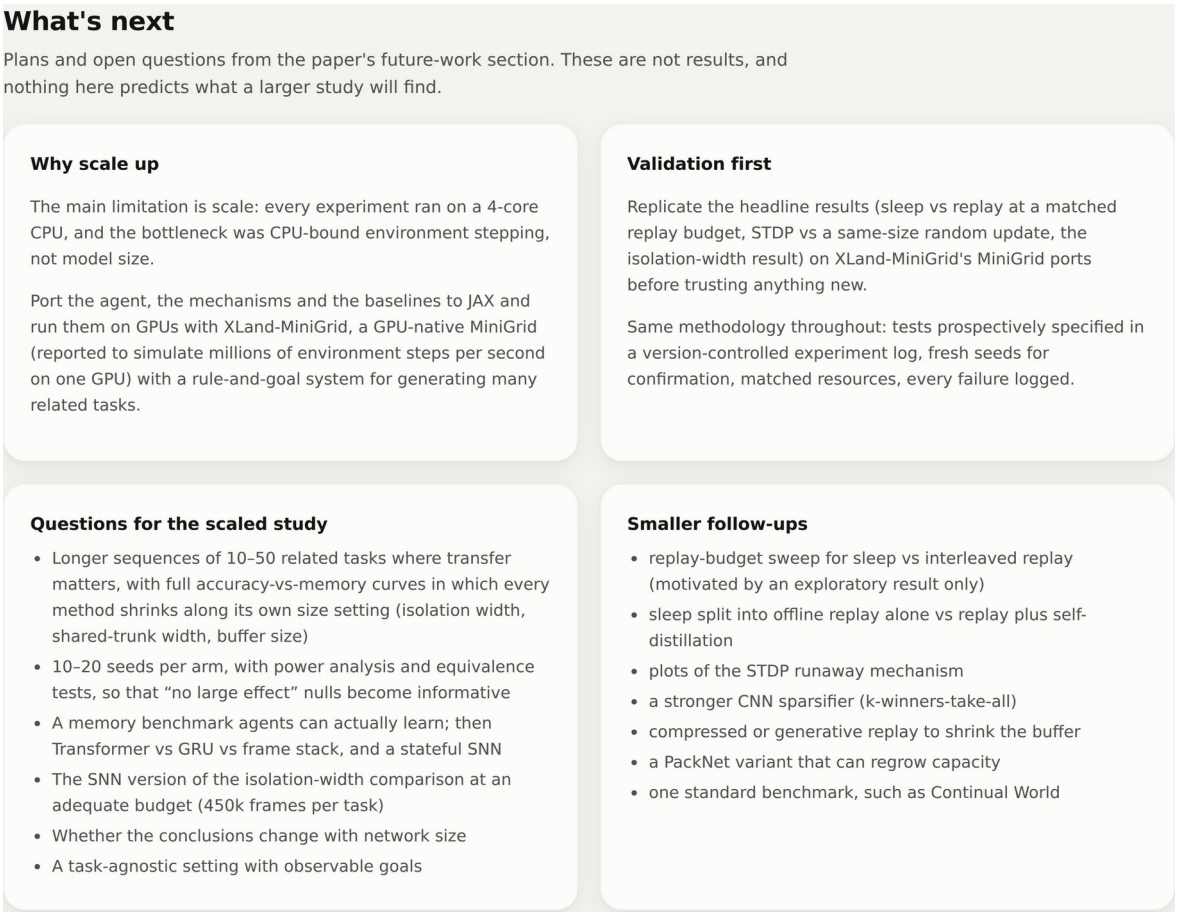


Fig A.4: Dashboard: the “What's next” section (plans, not results)

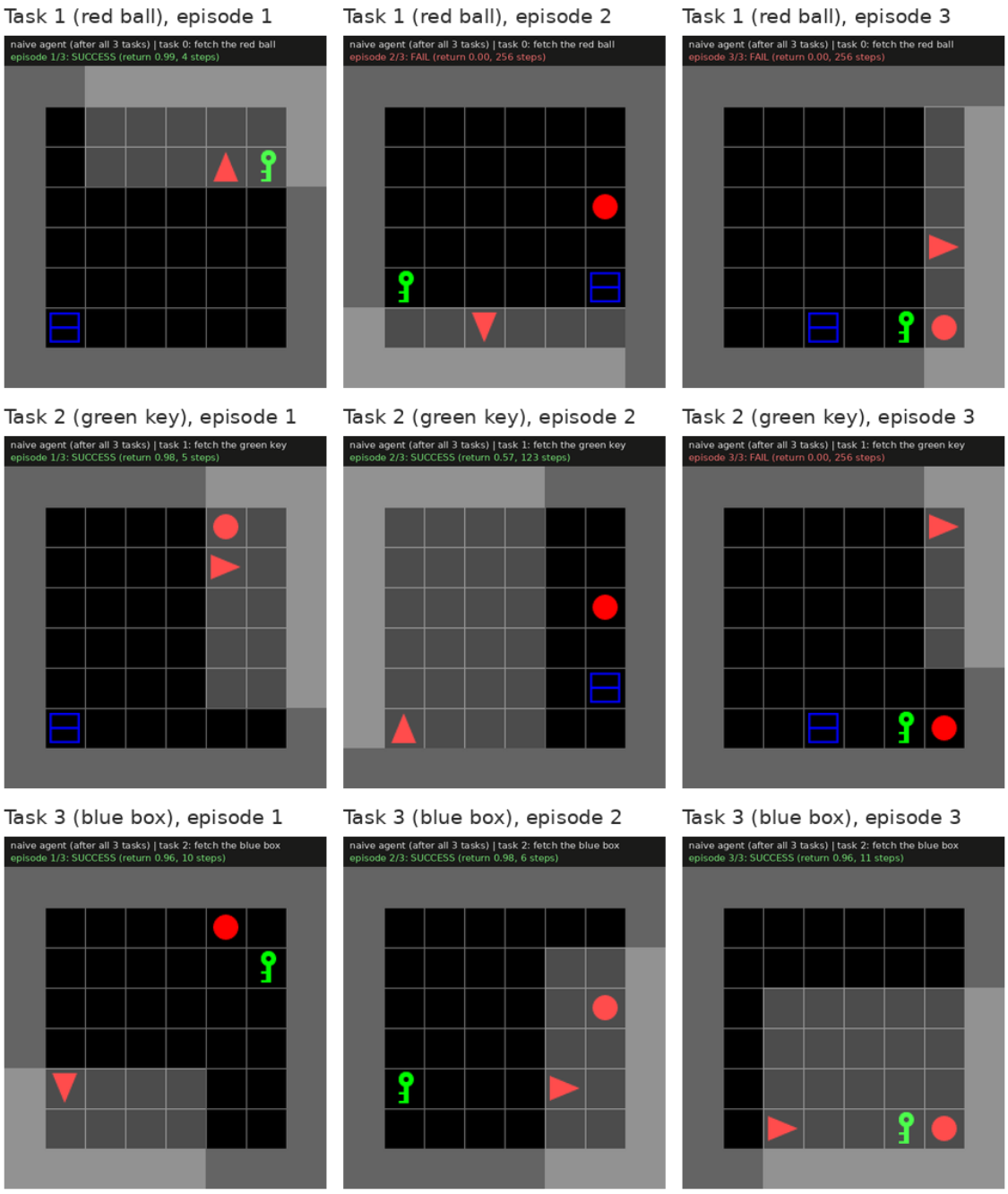


Fig A.5: Final frames of the three recorded episodes per task, naive fine-tuning agent after the full sequence (the episode outcome is printed above each frame)

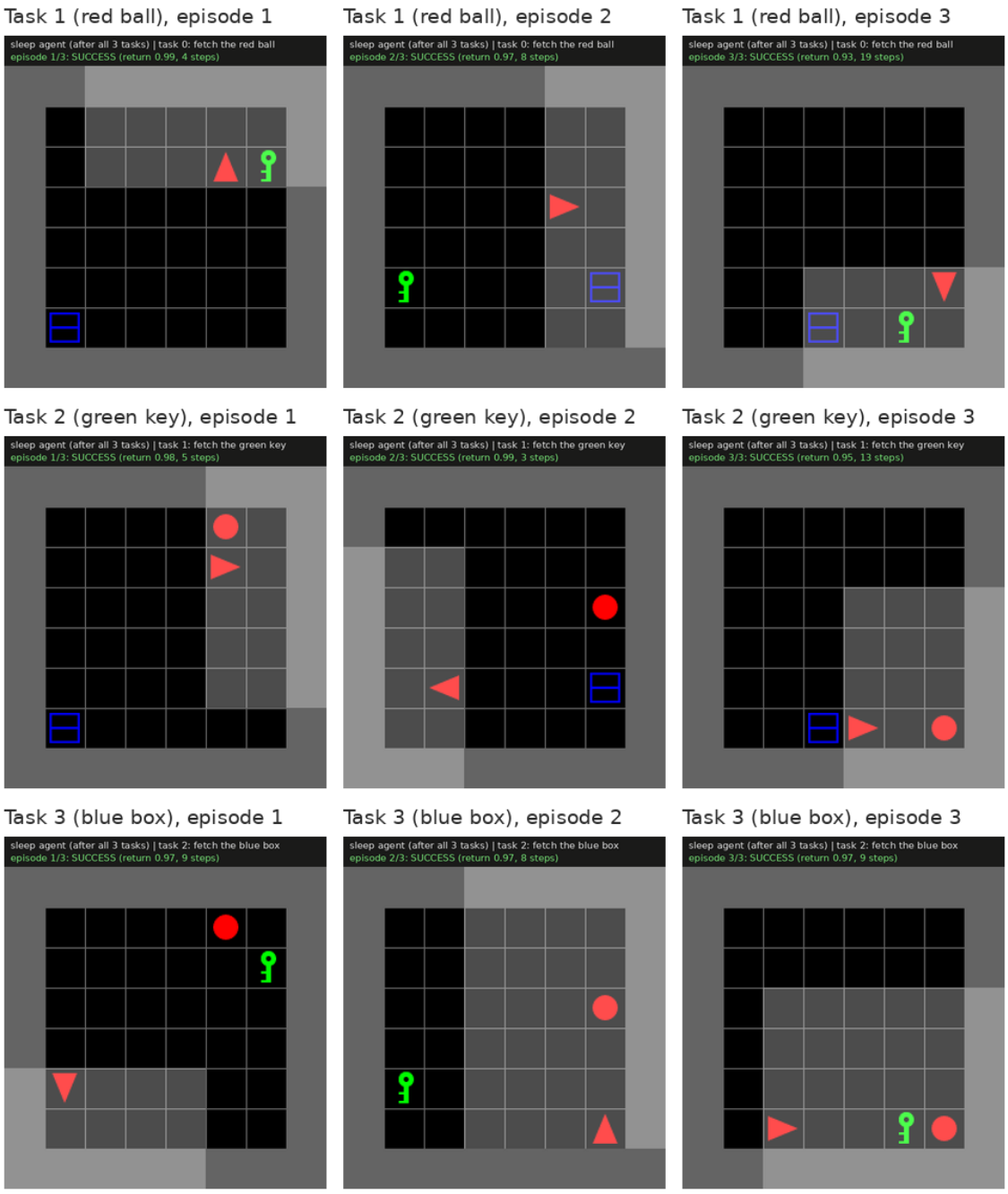


Fig A.6: Final frames of the three recorded episodes per task, sleep agent after the full sequence

Table A.1: Accuracy matrix of the naive fine-tuning demo agent (seed 1): mean return on each task (columns) after training each task (rows), 100 evaluation episodes per cell

	Task 1 (red ball)	**Task 2 (green key)**	**Task 3 (blue box)**
after task 1	0.977	0.275	0.119
after task 2	0.265	0.979	0.141
after task 3	0.126	0.542	0.956

Table A.2: Accuracy matrix of the sleep demo agent (seed 1): mean return on each task (columns) after training each task (rows), 100 evaluation episodes per cell

	Task 1 (red ball)	**Task 2 (green key)**	**Task 3 (blue box)**
after task 1	0.978	0.227	0.165
after task 2	0.979	0.980	0.080
after task 3	0.978	0.979	0.979

APPENDIX B: CODING

Key source excerpts only, copied verbatim from the repository; the complete code is at https://github.com/VT69/NeuroPlast. Each listing names the file it comes from.

Listing B.1: neuroplast/models/snn/lif.py (LIF neuron with a fast-sigmoid surrogate gradient)

```
class FastSigmoidSpike(torch.autograd.Function):
    @staticmethod
    def forward(ctx, v_minus_thr, slope):
        ctx.save_for_backward(v_minus_thr)
        ctx.slope = slope
        return (v_minus_thr > 0).to(v_minus_thr.dtype)

    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        grad = grad_out / (1.0 + ctx.slope * x.abs()) ** 2
        return grad, None


def spike_fn(v_minus_thr: torch.Tensor, slope: float = 25.0) -> torch.Tensor:
    return FastSigmoidSpike.apply(v_minus_thr, slope)


class LIF(nn.Module):
    """Stateless-per-call LIF: caller threads the membrane state through time.

    beta is optionally learnable (parameterised via a sigmoid so it stays in
    (0, 1)); per-channel if `channels` is given.
    """

    def __init__(self, beta: float = 0.9, threshold: float = 1.0, slope: float = 25.0,
                 learn_beta: bool = False, channels: int | None = None):
        super().__init__()
        self.threshold = threshold
        self.slope = slope
        init = torch.logit(torch.tensor(float(beta)))
        shape = (channels,) if channels else ()
        if learn_beta:
            self.beta_logit = nn.Parameter(init.expand(shape).clone())
        else:
            self.register_buffer("beta_logit", init.expand(shape).clone())

    def beta(self, ndim: int) -> torch.Tensor:
        b = torch.sigmoid(self.beta_logit)
        if b.ndim == 1 and ndim == 4:  # per-channel for conv maps (B, C, H, W)
            b = b.view(1, -1, 1, 1)
        return b

    def forward(self, current: torch.Tensor, v: torch.Tensor | None, s_prev:
        torch.Tensor | None):
        if v is None:
            v = torch.zeros_like(current)
            s_prev = torch.zeros_like(current)
        # reset term is detached (as in snnTorch's default) for stable gradients
        v = self.beta(current.ndim) * v + current - self.threshold * s_prev.detach()
        s = spike_fn(v - self.threshold, self.slope)
        return s, v
```

Listing B.2: neuroplast/learning/stdp.py (pair-based STDP with eligibility, dense layers)

```
def traces(seq: Sequence[torch.Tensor], lam: float) -> list[torch.Tensor]:
    """Exponential trace over a list of spike tensors (inclusive of step t)."""
    out, tr = [], None
    for s in seq:
        tr = s if tr is None else lam * tr + s
        out.append(tr)
    return out


def _mod(mod, B, ref):
    if mod is None:
        return ref.new_ones(B)
```

```
    return mod.to(ref.dtype)


def stdp_dense(pre: Sequence[torch.Tensor], post: Sequence[torch.Tensor], modulator:
    torch.Tensor | None = None,
               a_plus=1.0, a_minus=1.0, lam_pre=0.8, lam_post=0.8, mean=True) ->
                   torch.Tensor:
    """pre[t]: (B, in), post[t]: (B, out) spikes. Returns dW (out, in).

    dW = sum_b M_b sum_t [A+ post_b[t] x_b[t]^T - A- y_b[t-1] pre_b[t]^T]  (/B if mean)
    """
    B = pre[0].shape[0]
    m = _mod(modulator, B, pre[0])[:, None]
    x = traces(pre, lam_pre)
    y = traces(post, lam_post)
    dW = pre[0].new_zeros(post[0].shape[1], pre[0].shape[1])
    for t in range(len(pre)):
        dW += a_plus * (m * post[t]).T @ x[t]
        if t > 0:
            dW -= a_minus * (m * y[t - 1]).T @ pre[t]
    return dW / B if mean else dW
```

Listing B.3: neuroplast/learning/hybrid.py (hybrid rule: STDP or random update, homeostasis)

```
    @torch.no_grad()
    def after_step(self, agent, batch, update):
        self._n += 1
        rec = self.enc.record
        if not rec:
            return
        if self.homeo is not None:
            self._homeostasis(rec)
        if self.alpha == 0 or self._n % self.every:
            return
        n_frames = rec[0][0][0].shape[0]
        mod = self._modulator(batch, n_frames, agent.window)
        if isinstance(mod, str):
            self.skipped = getattr(self, "skipped", 0) + 1
            return
        for i in self.idx:
            layer = self.enc.layers[i]
            pre, post = rec[i]
            if self.center:
                pre, post = _center(pre), _center(post)
            if self.mode == "random":
                dW = torch.randn_like(layer.weight)
            elif i < 3:
                dW = stdp_conv(pre, post, layer.kernel_size, mod, **self.kw)
            else:
                dW = stdp_dense(pre, post, mod, **self.kw)
            rms = dW.pow(2).mean().sqrt()
            self.last_rms[i] = rms.item()
            if rms < 1e-12:
                continue
            if self.mode == "hebbian":
                w = layer.weight
                norms = w.flatten(1).norm(dim=1)
                w += (self.alpha * self.lr / rms) * dW
                scale = norms / w.flatten(1).norm(dim=1).clamp_min(1e-12)
                w *= scale.view(-1, *([1] * (w.dim() - 1)))
            else:
                layer.weight += (self.alpha * self.lr / rms) * dW

    @torch.no_grad()
    def _homeostasis(self, rec):
        """Local intrinsic plasticity: nudge each neuron's bias toward a target rate."""
        for i in self.idx:
            post = torch.stack(rec[i][1])  # (T, B, C[, H, W])
            dims = (0, 1) + tuple(range(3, post.dim()))
            rate = post.mean(dim=dims)
            self.enc.layers[i].bias += self.homeo_lr * (self.homeo - rate)
```

Listing B.4: neuroplast/baselines/replay.py (CLEAR-style replay and the distillation loss)

```
def distill_loss(agent, obs, mask, task, logits_t, value_t, vf_coef=0.5):
    logits, value = agent(obs, mask, task)
    kl = F.kl_div(F.log_softmax(logits, -1), F.log_softmax(logits_t, -1),
        log_target=True,
```

```
                  reduction="batchmean")
    return kl + vf_coef * F.mse_loss(value, value_t)


class _ReplayHook:
    def __init__(self, owner):
        self.owner = owner
        self.buf, self.batch, self.coef = owner.buf, owner.batch, owner.coef

    def extra_loss(self, agent, batch):
        self.owner.replay_samples += self.batch
        o, m, t, l, v = self.buf.sample(self.batch)
        return self.coef * distill_loss(agent, o, m, t, l, v)


class Replay:
    def __init__(self, buffer_per_task=5000, batch=128, coef=1.0):
        self.buf = TaskBuffer()
        self.buffer_per_task, self.batch, self.coef = buffer_per_task, batch, coef
        self.replay_samples = 0

    def hooks(self, agent, task_idx, cfg):
        return [_ReplayHook(self)] if len(self.buf) else []

    def end_task(self, agent, task_idx, cfg):
        o, m, l, v = collect_states(agent, cfg.env_id, task_idx, self.buffer_per_task)
        self.buf.add(o, m, task_idx, l, v)
```

Listing B.5: neuroplast/sleep/sleep.py (sleep phase)

```
    def sleep_phase(self, agent, task_idx):
        stdp = None
        was_recording = getattr(agent.encoder, "record_spikes", False)
        if self.stdp_alpha > 0 and hasattr(agent.encoder, "lifs"):
            stdp = HybridSTDP(alpha=self.stdp_alpha, mode="hebbian",
                layers=self.stdp_layers, lr=self.lr,
                               center=self.stdp_center, homeo=self.stdp_homeo)
            stdp.setup(agent)
        snapshot = make_snapshot(agent) if self.rehearsal_coef > 0 and self.reservoir.n
            else None
        opt = torch.optim.Adam(agent.parameters(), lr=self.lr)
        agent.train()
        tot = 0.0
        steps = self._phase_steps()
        for _ in range(steps):
            loss = 0.0
            if self.dream and stdp is not None:
                # dream: random binary frames with the stored mean input statistics,
                    STDP only
                x = (torch.rand(self.batch, *self.input_mean.shape) <
                    self.input_mean).float()
                with torch.no_grad():
                    agent.encoder(x)
                stdp.after_step(agent, dict(adv=None, mask=None), 0)
                if not self.distill:
                    continue
            if snapshot is not None:
                o2, m2 = self.reservoir.sample(self.batch)
                with torch.no_grad():
                    l2, v2 = snapshot(o2, m2, task_idx)
                loss = self.rehearsal_coef * distill_loss(agent, o2, m2, task_idx, l2,
                    v2)
            if self.distill:
                # last forward = old-task replay, so STDP below sees old-task spikes
                o, m, t, l, v = self.buf.sample(self.batch)
                loss = loss + distill_loss(agent, o, m, t, l, v)
                self.replay_samples += self.batch
            if isinstance(loss, float):
                continue
            opt.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(agent.parameters(), 0.5)
            opt.step()
            tot += loss.item()
            if stdp is not None and not self.dream:
                stdp.after_step(agent, dict(adv=None, mask=None), 0)
        if stdp is not None:
            agent.encoder.record_spikes = was_recording  # a wake-time STDP hook may
                need it
```

```
            agent.encoder.record = []
        self.n_phases += 1
        return dict(sleep_loss=tot / max(steps, 1))
```

Listing B.6: neuroplast/eval/metrics.py (continual-learning metrics)

```
def cl_metrics(R, curves=None, ref_curves=None):
    R = np.asarray(R, dtype=float)
    T = R.shape[0]
    out = dict(ACC=float(R[-1].mean()))
    if T > 1:
        out["BWT"] = float(np.mean([R[-1, j] - R[j, j] for j in range(T - 1)]))
        out["FORGET"] = float(np.mean([R[:T - 1, j].max() - R[-1, j] for j in range(T -
            1)]))
    if curves is not None and ref_curves is not None:
        fwt = []
        for c, rc in zip(curves, ref_curves):
            a, ar = auc(c), auc(rc)
            fwt.append((a - ar) / (1 - ar) if ar < 1 else 0.0)
        out["FWT"] = float(np.mean(fwt[1:])) if len(fwt) > 1 else 0.0
        out["FWT_per_task"] = [float(x) for x in fwt]
    return out


def auc(curve):
    """Mean of a return curve (list of returns at evenly spaced checkpoints)."""
    c = np.nan_to_num(np.asarray(curve, dtype=float), nan=0.0)
    return float(c.mean()) if len(c) else 0.0
```

Listing B.7: scripts/memory_fair.py (memory accounting)

```
OBS_B, FIXED_B = 147, 40          # per frame; mask/window adds K bytes
RESERVOIR = 5000                  # sleep's current-task reservoir
    (neuroplast/sleep/sleep.py)


def bytes_per_state(window):
    return OBS_B * window + window + FIXED_B


def load():
    groups = defaultdict(list)
    for root, trunk in ROOTS.items():
        for p in glob.glob(os.path.join(root, "*", "results.json")):
            r = json.load(open(p))
            if not r.get("metrics"):
                continue
            method = r["method"]
            name = os.path.basename(os.path.dirname(p)).rsplit("_s", 1)[0]
            cfg = r["cfg"]
            window = cfg.get("window", 1) if cfg.get("memory") else 1
            T = len(r["tasks"])
            mk = r.get("method_kwargs", {}) or {}
            buf_per_task = mk.get("buffer_per_task", 5000) if method in ("replay",
                "sleep") else 0
            if method == "sleep" and mk.get("distill") is False:
                buf_per_task = 0
            bps = bytes_per_state(window)
            params_mb = r["params"] * 4 / 1e6
            buf_mb = buf_per_task * T * bps / 1e6
            peak_extra = (min(RESERVOIR, 10**9) * bps + r["params"] * 4) / 1e6 if method
                == "sleep" else 0.0
            name = f"{name} @{cfg['total_frames'] // 1000}k"  # 150k vs 450k fetch5 runs
                share dir names
            groups[(r["suite"], trunk, name)].append(dict(
                acc=r["metrics"]["ACC"], params_mb=params_mb, buf_mb=buf_mb,
                    total_mb=params_mb + buf_mb,
                impl_mb=params_mb + 2 * buf_mb, peak_mb=params_mb + buf_mb + peak_extra,
                buf_per_task=buf_per_task, method=method, bps=bps, T=T,
                    frames=cfg["total_frames"]))
    return groups
```

Listing B.8: paper/compute_numbers.py (confidence intervals and tests)

```
def welch(a, b):
    a, b = np.asarray(a, float), np.asarray(b, float)
```

```
    d = a.mean() - b.mean()
    va, vb = a.var(ddof=1) / len(a), b.var(ddof=1) / len(b)
    se = math.sqrt(va + vb)
    if se == 0:
        return d, d, d, 0.0
    df = (va + vb) ** 2 / (va ** 2 / (len(a) - 1) + vb ** 2 / (len(b) - 1))
    t = stats.t.ppf(0.975, df)
    return d, d - t * se, d + t * se, stats.ttest_ind(a, b, equal_var=False).pvalue


def wilson(k, n, z=1.959964):
    p = k / n
    den = 1 + z * z / n
    c = (p + z * z / (2 * n)) / den
    h = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / den
    return c - h, c + h


def newcombe(k1, n1, k2, n2):
    p1, p2 = k1 / n1, k2 / n2
    l1, u1 = wilson(k1, n1)
    l2, u2 = wilson(k2, n2)
    d = p1 - p2
    return d, d - math.sqrt((p1 - l1) ** 2 + (u2 - p2) ** 2), d + math.sqrt((u1 - p1) **
        2 + (p2 - l2) ** 2)


def fisher(k1, n1, k2, n2):
    return stats.fisher_exact([[k1, n1 - k1], [k2, n2 - k2]]).pvalue
```

REFERENCES

[1] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, "Towards continual reinforcement learning: A review and perspectives," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1401–1476, 2022.

[2] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[3] D. Rolnick, A. Ahuja, J. Schwarz, T. P. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[4] A. Mallya and S. Lazebnik, "PackNet: Adding multiple tasks to a single network by iterative pruning," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7765–7773.

[5] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen et al., "SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, art. no. eadi1480, 2023.

[6] C. Zhou, L. Yu, Z. Zhou, H. Zhang, J. Wang, H. Zhou et al., "Spikingformer: A key foundation model for spiking neural networks," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 40, no. 3, 2026, pp. 2236–2244.

[7] C. Lee, Y. Park, S. Yoon, J. Lee, Y. Cho, and C. Park, "Brain-inspired learning rules for spiking neural network-based control: A tutorial," *Biomedical Engineering Letters*, vol. 15, pp. 37–55, 2025.

[8] B. Huo, F. Li, S. Peng, H. Chen, S. Xin, and H. Wang, "Research on SNN learning algorithms and networks based on biological plausibility," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3566717.

[9] T. Tadros, G. P. Krishnan, R. Ramyaa, and M. Bazhenov, "Sleep-like unsupervised replay reduces catastrophic forgetting in artificial neural networks," *Nature Communications*, vol. 13, art. no. 7742, 2022.

[10] G. G. Turrigiano and S. B. Nelson, "Homeostatic plasticity in the developing nervous system," *Nature Reviews Neuroscience*, vol. 5, no. 2, pp. 97–107, 2004.

[11] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou et al., "Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 36, 2023.

[12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[13] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.

[14] H. Yu, F. Zhu, G.-S. Xie, and L. Shao, "Self-consolidation for self-evolving agents," arXiv:2602.01966, 2026.

[15] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[16] K. Van den Berghe, S. Stroobants, V. J. Reddi, and G. C. H. E. de Croon, "Adaptive surrogate gradients for sequential reinforcement learning in spiking neural networks," in

Advances in Neural Information Processing Systems, vol. 38, 2025.

[17] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi et al., "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[18] V. Pandey and D. Biswas, "Spiking decision transformers: Local plasticity, phase-coding, and dendritic routing for low-power sequence control," arXiv:2508.21505, 2025.

[19] G. Bi and M. Poo, "Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type," *Journal of Neuroscience*, vol. 18, no. 24, pp. 10464–10472, 1998.

[20] N. Frémaux and W. Gerstner, "Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules," *Frontiers in Neural Circuits*, vol. 9, art. no. 85, 2016.

[21] A. Nøkland, "Direct feedback alignment provides learning in deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[22] M. Wołczyk, M. Zając, R. Pascanu, Ł. Kuciński, and P. Miłoś, "Continual World: A robotic benchmark for continual reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[23] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proc. European Conference on Computer Vision (ECCV)*, 2018, pp. 532–547.

[24] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *Proc. IEEE International Solid-State Circuits Conference (ISSCC)*, 2014, pp. 10–14.

[25] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. M. Araújo, "CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms," *Journal of Machine Learning Research*, vol. 23, no. 274, pp. 1–18, 2022.

[26] R. G. Newcombe, "Interval estimation for the difference between independent proportions: Comparison of eleven methods," *Statistics in Medicine*, vol. 17, no. 8, pp. 873–890, 1998.

[27] B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," *Biometrika*, vol. 34, no. 1/2, pp. 28–35, 1947.

[28] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[30] A. Nikulin, V. Kurenkov, I. Zisman, A. Agarkov, V. Sinii, and S. Kolesnikov, "XLand-MiniGrid: Scalable meta-reinforcement learning environments in JAX," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 37, 2024.

[31] H. Shin, J. K. Lee, J. Kim, and J. Kim, "Continual learning with deep generative replay," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.